

TFG - Ingeniería del software

Autor:

Jose Luis Tresserras Merino

Director:

Santi Caballe Llobet

Tutor:

Oriol Martí Girona

TFRM Scheduler



Licencia

Copyright (C) 2024 JOSE LUIS TRESSERRAS MERINO.

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is included in the section entitled "GNU Free Documentation License".

Ficha del trabajo final

Título del trabajo:	TFRM Scheduler
Nombre del autor:	Tresserras Merino, Jose Luis
Nombre del consultor/a	Martí Girona, Oriol
Fecha de entrega (mm/aaaa):	06/2024
Titulación:	Ingeniería Informática
Área del Trabajo Final:	Ingeniería del software
Idioma del trabajo:	Castellano
Palabras clave:	scheduler, linux, planificación, OR-Tools, observatorio
Resumen del Trabajo (máximo 250 palabras): Con la finalidad, contexto de aplicación, metodología, resultados y conclusiones del trabajo.	
Realizando unas prácticas junto al Observatorio Fabra, se me planteó la oportunidad de realizar la planificación de las observaciones de manera automatizada como trabajo final de grado. A lo largo de este proyecto, he tenido la oportunidad de obtener nuevos conocimientos sobre las bases de las planificaciones, y realizar un estudio en profundidad. Ha sido muy interesante y me ha ayudado a comprender mejor cómo abordar los problemas de planificación. Dado que al comenzar el proyecto, no tenía ningún conocimiento práctico en el campo de las planificaciones, he invertido mucho tiempo estudiando los diferentes planificadores y las técnicas que utilizan. Me ha gustado poder ver implementaciones prácticas de los ejercicios de lógica realizados durante el grado, ha sido muy interesante. He escogido realizar la metodología ágil para la implementación del proyecto dado mi desconocimiento sobre el proceso de estudio. Al final los tiempos planificados y las implementaciones han sido muy precisas y no he requerido de cambios en las fechas de entrega. He podido realizar la implementación de gran parte del trabajo, incluso desarrollar una demo que permite mostrar un ejemplo del funcionamiento de una planificación básica. ¡Incluso se ha podido probar en el observatorio! Estoy muy contento con el resultado obtenido y espero que sea de ayuda para el observatorio. Me gustaría poder realizar más aportaciones para que el proyecto pueda obtener mejores resultados en el futuro.	

Abstract (in English, 250 words or less):

This project is trying to solve a scheduling problem. In collaboration with the Observatorio Fabra, I have developed an application that is helping the observatory save time making the planifications of the objects to schedule at night. The main focus of this project was to start gathering all the possible information to get a better comprehension of the necessary approach to solve scheduling problems and making a demo to test the viability of the project.

Scheduling has been found to be a very difficult problem. It is a problem that is currently unsolvable, because it can have almost infinite solutions. And it is also very difficult to establish all the requirements and set good enough constraints to the problems you are trying to solve.

My solution is focusing on the ability to establish as many constraints as you can, so the solution can be much more narrowed to the goal. This project wants to be able to offer a simple interface to establish dynamic constraints depending on the result you want to obtain. We manage to get that thanks to a module system that is able to toggle different pre-programmed rules based on the needs of the result.

In conclusion, the goals have been met and the observatory has been able to perform some tests with real data, and relieve some work in some static observations thanks to the predefined constraints. We will continue with the development of the project and introduce new ways to establish constraints.

Quiero dar un enorme agradecimiento a mi familia, por haberme apoyado durante este largo camino, animándome a continuar cada paso.

Muchas gracias.

Índice

Plan de trabajo	1
1. Introducción	1
1.1 Contexto y justificación	1
2. Objetivos	2
3. Alcance	3
4. Funcionalidades	4
4.1 Requisitos funcionales	4
4.2 Requisitos no funcionales	4
5. Metodología de desarrollo	6
6. Planificación	7
• 6.1 Primera iteración	7
6.1.1 Primer tramo	7
• 6.2 Segunda iteración	8
6.2.1 Segundo tramo	8
6.2.2 Tercer tramo	8
6.2.3 Cuarto tramo	9
• 6.3 Tercera iteración	9
6.3.1 Quinto tramo	9
6.3.2 Sexto tramo	10
6.3.3 Séptimo tramo	11
6.3.4 Octavo tramo	11
Análisis de Planificadores	13
7. ¿Qué es una planificación óptima?	13
8. Soluciones para planificaciones	14
8.1 Algoritmos de búsqueda	14
8.2 Problemas de satisfacción de restricciones (CSP)	15
8.3 Definiciones	15
9. Prueba de concepto	18
10. Herramientas	20
Implementación	22
11. Análisis	22
11.1 Requisitos	22
11.2 Funcionalidades descartadas	23
11.3 Funcionalidades opcionales	24
11.4 Descripción de datos	25
11.5 Historias de usuario	26
11.6 Casos de uso	30
11.7 Planes de Test	35

12. Diseño	36
12.1 Diagrama de clases	36
12.2 Modelo entidad-relación	37
13. Arquitectura	40
13.1 DevOps	40
13.2 Gestión de la configuración	40
13.3 Trazabilidad	40
13.4 Modularidad	41
13.5 Testing	41
14. Gestión	42
14.1 Estudio de mercado	42
15. Programación	45
15.1 Lenguaje de programación	45
15.2 Estructura del proyecto	46
15.3 Dependencias	47
15.4 Implementación producto final	48
15.5 Ejemplos de ejecución	49
17. Entrega y seguimiento	51
Documentación de la entrega	51
Conclusiones	52
Referencias	53
18. Glosario	53
19. Bibliografía	53
GNU Free Documentation License	55

Lista de figuras

Google docs no soporta las figuras, para la PEC3, antes de realizar la entrega, las insertaré con LibreOffice.

Plan de trabajo

1. Introducción

Este documento detalla el plan de trabajo para el desarrollo e implementación de un proyecto en colaboración con el Observatorio Fabra de Barcelona, en concreto con el uso remoto y robótico del telescopio TFRM (Telescopio Fabra-ROA en el Montsec). El proyecto busca la implementación de un planificador automatizado para organizar las observaciones que se han de realizar durante la noche, realizando un reparto optimizado de las observaciones proporcionadas sobre unos campos específicos para cada proyecto de observación.

1.1 Contexto y justificación

El telescopio TFRM forma parte de un proyecto conjunto entre dos instituciones, la Real Academia de Ciencias y Artes (Observatorio Fabra en Barcelona) y el Real Instituto y Observatorio de la Armada de San Fernando (Cádiz). La principal funcionalidad del telescopio es realizar seguimientos fotográficos a satélites artificiales. Gracias a su óptica modificada, el telescopio puede realizar observaciones de gran calidad y magnitud estelares. En buenas condiciones meteorológicas, tiene una magnitud estelar de entre 20 y 21. Gracias a estas prestaciones y más, el telescopio tiene un gran abanico de posibles trabajos a realizar.

Todas las tareas de mantenimiento y observación se han repartido entre los miembros encargados del telescopio. Los equipos cuentan con profesionales astrofísicos para realizar las observaciones y tareas del mantenimiento de servidores y el procesamiento de los datos. Sin embargo, el equipo de observación cree que se pueden automatizar las tareas de observación, y ahorrar mucho tiempo realizando la planificación de las observaciones.

2. Objetivos

El objetivo principal de este proyecto, es optimizar el tiempo que le toma al encargado de la observación, planificar las tareas que ha de realizar el telescopio durante la noche. Por ello, se quiere buscar una solución que consiga utilizar el máximo uso de la información disponible, aumentar la eficiencia del telescopio y reducir drásticamente el tiempo necesario de la planificación para el observador.

Para realizar estas tareas se necesita crear una infraestructura que permita modificar y consultar los datos con los que va a interactuar nuestra aplicación.

Por último, se quiere entregar la aplicación a través de una licencia primitiva que ayude a un desarrollo abierto. Dónde se permita a los usuarios estudiar, modificar y utilizar el código compartiendo todo el conocimiento. Por ello se quiere garantizar la accesibilidad al código fuente, permitiendo que sea accesible para todo el mundo y se pueda promover la colaboración e innovación.

Se busca una implementación a medida para el observatorio que permita reducir el tiempo que conlleva la planificación de los objetos a observar por el telescopio. Por ello, es necesario tener una relación cercana con los técnicos que van a utilizar el software. De esta manera se va a poder compartir el desarrollo realizado en pequeñas iteraciones y recopilar comentarios que ayuden a mejorar constantemente el desarrollo del producto final.

3. Alcance

El alcance de este proyecto es principalmente el desarrollo del planificador, que pueda obtener los datos necesarios para poder realizar la planificación de múltiples coordenadas a lo largo de una noche. Se quiere conseguir una implementación que permita mejorar drásticamente el tiempo que conlleva realizar la planificación actualmente. Además, el producto final ha de ser fácil de extender con nuevas restricciones que se quieran ir añadiendo en el futuro.

Para mantener el enfoque en los objetivos principales de este proyecto, queda fuera del alcance el diseño adicional, cualquier tipo de interfaz gráfica que no se requiera para el correcto funcionamiento del planificador. Se espera que los usuarios tengan conocimientos avanzados y no utilicen interfaces gráficas específicas para el mantenimiento de los datos.

La mayor limitación de este proyecto, es la capacidad de cálculo para obtener la planificación más óptima. Los problemas de planificación son problemas NP-Complejos, eso quiere decir que analizar todas las variaciones de todas las posibles soluciones puede llegar a ser muy costoso en tiempo y recursos de la máquina donde se ejecuta. Además, puede haber más de una solución que cumpla con todas las limitaciones que nos ha establecido el observador. Por ello, en algunos casos concretos un humano ha de poder intervenir manualmente para asegurar que se escoge la mejor variación propuesta. Sin embargo, se considera que el trabajo de simplemente escoger la planificación deseada, es mucho más eficiente en comparación con el método de trabajo utilizado actualmente.

4. Funcionalidades

4.1 Requisitos funcionales

Planificación de la observación

El sistema debe proporcionar una herramienta que permita programar y organizar las tareas de observación a lo largo de una noche. Este debe tener en cuenta diversos factores que pueden influir en la organización de las tareas, y ha de poder resolver los conflictos para devolver una planificación exitosa. Además, el programa debe generar alertas de aviso en caso de no encontrar una solución adecuada.

Tratamiento de los datos

Una vez realizada la observación, en caso de ser exitosa y aprobada por el observador, esta ha de poder ser exportada con formato XML. Se busca poder establecer una comunicación entre varios programas internos para una mayor optimización del empleo de la herramienta.

4.2 Requisitos no funcionales

Rendimiento

El planificador ha de ser capaz de ingerir una gran cantidad de datos, que le permita tener en cuenta todas las variables posibles. Ha de poder tratar todos los datos y dar un resultado en el menor tiempo posible.

Aproximación modular

En caso de que se quieran añadir nuevos requisitos de planificación, el programa ha de estar bien preparado para que sea sencillo de adaptar. Lo ideal sería realizar un administrador de requisitos que facilite añadir o quitar funcionalidades sin la necesidad de modificar la estructura del código.

Integración

El sistema dónde se ejecutará este programa no necesita tener acceso al exterior, y debe ser ejecutado localmente por máquinas con una gran potencia de cálculo. Por estos motivos ha de poder establecer una comunicación con servicios internos que le brinden toda la información necesaria para cumplir su función

Soporte

Una vez realizada la totalidad del proyecto, el programa final ha de disponer únicamente de mantenimiento de emergencia. No se tiene contemplado realizar un mantenimiento periódico o seguido de la estabilidad del programa. Una vez entregado ha de estar meticulosamente probado y validado, para garantizar su fiabilidad para una funcionalidad óptima.

Legales

La totalidad del proyecto ha de tener una licencia primitiva que permita a todas las personas estudiar, modificar y utilizar el software sin ningún tipo de limitación o restricción alguna.

5. Metodología de desarrollo

Para el desarrollo de esta aplicación se va a utilizar una metodología iterativa. Se ha escogido este tipo de metodología debido a sus cortas etapas y la entrega continua de los resultados obtenidos. Se cree, que el proyecto se puede beneficiar de una entrega incremental al usuario para obtener una retroalimentación temprana que permita realizar ajustes mínimos en problemas inesperados o en caso de dificultades. De esta manera se pueden centrar todos los recursos en la mejora del producto final.

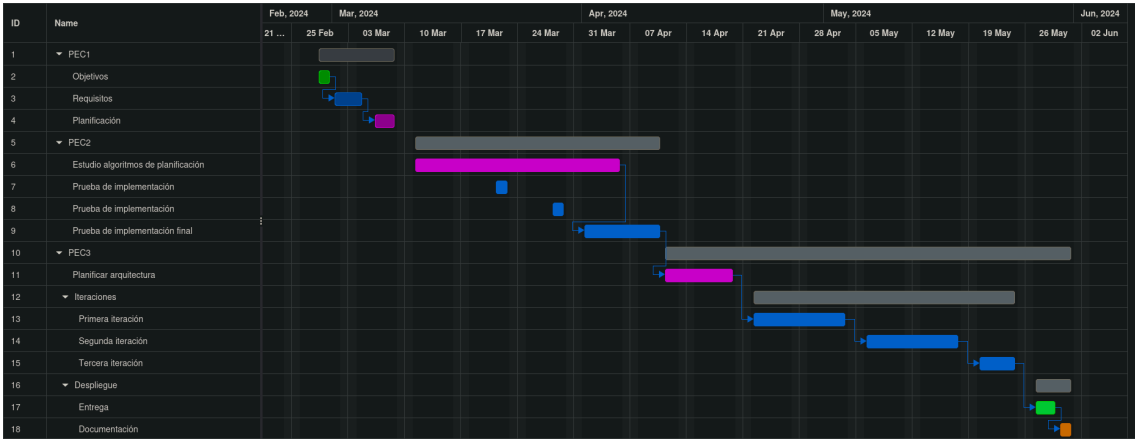
En esta primera etapa del proyecto, se han logrado establecer los objetivos necesarios para obtener un producto final que satisfaga las necesidades y expectativas del usuario. Además, se ha conseguido fijar unos requisitos básicos que ayudan a mantener una meta firme para desarrollar nuestra aplicación. Finalmente, se ha planificado el resto del tiempo necesario para completar el proyecto utilizando una metodología iterativa, que permite un desarrollo más veloz, con una mejor claridad de los objetivos, y el tiempo requerido para su finalización.

En la siguiente etapa se busca realizar el estudio sobre la teoría de los organizadores de tareas y demás algoritmos de planificación. Se quiere obtener un conocimiento profundo del funcionamiento de los algoritmos que permiten generar las planificaciones necesarias para el proyecto. Además, se pretende realizar la implementación de algunos casos de prueba que permitan establecer una idea inicial de la complejidad del proyecto y su estructura, para adquirir una idea sobre la magnitud del proyecto antes de la finalización de la etapa.

En la tercera y última etapa del proyecto, mediante la información obtenida en la fase anterior, es posible empezar a establecer la estructura necesaria para realizar la implementación de la arquitectura requerida del proyecto. Se quiere priorizar el desarrollo del producto final, para lograr obtener un mínimo producto viable que permita completar los objetivos y requisitos definidos. Además, se pretende realizar la entrega del producto al usuario final y poder documentar su funcionamiento en el telescopio.

6. Planificación

El siguiente diagrama de Gantt ilustra el tiempo esperado para cada etapa del proyecto separado en sus fases. Cada iteración o prueba de implementación se tiene que tener pensado las tareas que se van a realizar. Esto significa que se necesita tomar un poco de tiempo de la tarea anterior para preparar el contenido a estudiar o realizar en el siguiente paso. Además, se tiene que realizar la documentación de cada proceso realizado. Sin embargo, gracias a la metodología de desarrollo escogida ya se tiene contemplado realizar la planificación, implementación y documentación de cada iteración. De esta manera se pueden tener claros los objetivos a seguir desde el inicio, y realizar pequeños ajustes en caso de necesitarlos.



- 6.1 Primera iteración

6.1.1 Primer tramo

Planificación

Al finalizar esta iteración, se ha de poder empezar a realizar el estudio de los algoritmos y métodos de búsqueda.

Diseño del trabajo

Se ha preparado un plan de estudio y realización de una duración de aproximadamente unas 12 semanas más. Este plan contempla el estudio de las diversas maneras de realizar las planificaciones y el desarrollo de la aplicación.

Retroalimentación

Los objetivos y requisitos establecidos son correctos para el funcionamiento final de la aplicación. Se ha dejado un poco de margen para cambios en base del estudio sobre las planificaciones. El ciclo iterativo ha sido una buena elección para estudiar y planificar el trabajo que permite adaptarse durante el desarrollo.

- 6.2 Segunda iteración

6.2.1 Segundo tramo

Planificación

En este tramo voy a poder encontrar los principales algoritmos de búsqueda que se quieren comparar y realizar un estudio.

Diseño de trabajo

Durante este tramo se va a realizar principalmente el estudio sobre los tipos de algoritmos que existen para realizar planificaciones eficientemente. Se busca realizar el estudio de la información encontrada y resumir el conocimiento adquirido en el apartado de estudio.

Retroalimentación

Se ha conseguido unos conocimientos básicos del funcionamiento de los planificadores y se han descubierto sus principales métodos de funcionamiento.

6.2.2 Tercer tramo

Planificación

En el tramo final del estudio, se va a poder obtener los conocimientos del funcionamiento básico de los algoritmos principales. Se ha de poder encontrar la mejor opción para poder llevar a cabo el proyecto.

Diseño de trabajo

El principal objetivo es establecer unos conocimientos básicos que permitan realizar una elección válida a largo plazo de los algoritmos de planificación, por ello, se van a utilizar recursos menos detallados, pero de amplio espectro, que permitan obtener información global sobre el problema.

Retroalimentación

Se ha conseguido hacer una pequeña implementación para valorar los diferentes métodos de planificación y se ha tomado una decisión sobre la mejor metodología para este proyecto.

6.2.3 Cuarto tramo

Planificación

Se espera haber encontrado la herramienta a utilizar para este proyecto y tener claras las dependencias necesarias para empezar la implementación.

Diseño de trabajo

Durante este tramo se pretende encontrar diferentes utilidades que ayuden al desarrollo de la aplicación y realizar algunas pruebas que permitan realizar aproximaciones de los requisitos de la aplicación.

Retroalimentación

Se ha conseguido encontrar varias herramientas que facilitan las implementaciones de las planificaciones. Además, se ha podido escoger una y se va a poder empezar a realizar el modelo de la implementación. También se ha realizado la planificación de la tercera iteración.

- 6.3 Tercera iteración

6.3.1 Quinto tramo

Planificación

Al acabar este tramo, se ha de haber realizado la documentación de la estructura y funcionamiento del proyecto.

Diseño del trabajo

Durante este tramo, se pretenden estructurar todas las posibles funciones que puede realizar el usuario. Además de realizar la documentación de la estructura básica que ha de tener el programa final.

Análisis de los datos

Se ha de crear una base, con todas las clases necesarias para implementar la aplicación. Además, se ha de pensar en realizar la documentación de una estructura Entidad-Relación para en el futuro

Implementación

Se van a definir todos los objetivos necesarios para hacer una aplicación funcional. Además de los objetivos que no se van a implementar o algunos opcionales que se implementarán o no en el futuro.

Retroalimentación

Se ha podido empezar bien el proyecto y de momento no ha habido grandes contratiempos. Se espera con ganas poder empezar el desarrollo y ver la implementación terminada.

6.3.2 Sexto tramo

Planificación

Al acabar este tramo, se pretende estar listo para empezar a desarrollar la aplicación. Durante este tramo se ha de preparar el entorno de desarrollo, y realizar pruebas para acomodarse al entorno de desarrollo.

Diseño del trabajo

Se va a empezar a crear el proyecto y a realizar pruebas con OR-Tools. Además de documentar todo el proceso y establecer los requisitos que requiere la aplicación.

Retroalimentación

Los objetivos principales han sido bien recibidos y se espera con ansias poder empezar a desarrollar algunas pruebas que muestran un funcionamiento básico de la aplicación.

6.3.3 Séptimo tramo

Planificación

Al acabar este tramo, se pretende haber empezado con el desarrollo del producto y haber realizado la implementación de algunas funcionalidades básicas para enseñar al usuario final.

Diseño del trabajo

Se ha de empezar a realizar la implementación de la entrada a la aplicación y poder especificar todos los parámetros. Se ha de poder utilizar los parámetros de manera correcta sin fallos.

Análisis de los datos

Se han de poder tener en cuenta todos los posibles parámetros de entrada que ha de tener la aplicación.

Implementación

Cada parámetro ha de poder ser controlado y tener todos los posibles casos de uso implementados.

Retroalimentación

Se ha conseguido controlar todos los parámetros necesarios para poder utilizar la primera parte del ejecutable. El observatorio espera con ganas poder utilizar el programa completo. No se han añadido más funcionalidades y se está conforme con los planes de implementación.

6.3.4 Octavo tramo

Planificación

Al acabar este tramo, se espera poder ejecutar la aplicación con todas las funcionalidades. Se ha de poder realizar la entrega a los usuarios finales.

Diseño del trabajo

Se ha de realizar la implementación de las limitaciones al realizar las observaciones.

Implementación

Se espera haber realizado todas las implementaciones necesarias y haber tenido en cuenta todas las restricciones impuestas para realizar la planificación final. Se ha de poder entregar el ejecutable y realizar algunas pruebas en conjunto con los usuarios finales.

Retroalimentación

Se ha conseguido entregar una versión estable, sin embargo, sin todas las funcionalidades esperadas. Por el momento, el observador que utilice el programa va a utilizar el ejecutable y realizar cambios manuales si hay alguna previsión no contemplada por el software. Aún no haber podido entregar toda la aplicación, la recepción del programa ha sido satisfactoria y se está muy satisfecho con el método portable de ejecución.

Análisis de Planificadores

7. ¿Qué es una planificación óptima?

Una planificación óptima busca encontrar la mejor solución posible para alcanzar el objetivo definido, maximizando los recursos disponibles y minimizando los costes. Además, permite anticipar los resultados y adaptar los casos obtenidos según las necesidades. Una planificación óptima no requiere de encontrar la mejor solución posible para cada caso, dado que en muchas situaciones puede ser difícil o requerir mucho tiempo y recursos. Por ello, se trata de encontrar la solución más adecuada y eficiente dadas las limitaciones y restricciones del contexto en el que se desarrolla la planificación.

Las restricciones son unos pilares fundamentales, ya que definen los límites y condiciones necesarios para desarrollar las planificaciones. Sirven como guía para tomar decisiones, asegurando que los planes sean viables y alcanzables dentro de los límites establecidos. Las restricciones pueden ser de diversos tipos, como de tiempo, recursos, presupuesto, capacidades, etc. En este proyecto, por ejemplo, las restricciones de tiempo pueden determinar los plazos de las posiciones de las observaciones.

8. Soluciones para planificaciones

La planificación es un problema muy bien conocido, que lleva mucho tiempo siendo estudiado. Hay muchos tipos de planificaciones diferentes, y cada uno puede tener su propio método de resolución, según las tareas, recursos disponibles y objetivos planteados. Desde hace mucho tiempo, se ha intentado realizar la búsqueda de soluciones eficientes desde diferentes puntos de vista. Actualmente, hay dos grandes posibles rutas a escoger a la hora de buscar resolver un problema de optimización.

Los algoritmos de búsqueda, buscan, realizar una planificación utilizando las estrategias algorítmicas necesarias para generar posibles soluciones que propongan una solución factible. Por otro lado, utilizando la técnica de satisfacción de restricciones, se puede determinar la existencia de una solución para las planificaciones, si se adapta el problema, las restricciones y el resultado esperado de manera correcta, se puede determinar la existencia de una solución.

El estudio está centrado en los siguientes dos algoritmos, con el objetivo de comprender los métodos utilizados y sus beneficios.

8.1 Algoritmos de búsqueda

Hay infinitos tipos de problemas de planificación, y cada uno se puede beneficiar de un tipo concreto de algoritmo de búsqueda diferente para ser resuelto. Cada algoritmo ofrece una resolución óptima adaptada a sus circunstancias y objetivos definidos. El principal objetivo de este método, es encontrar la solución óptima para el problema descrito con el resultado esperado. Sin embargo, la solución óptima de dichos algoritmos puede ser muy costosa de encontrar. Esto es debido a la necesidad de iterar sobre los datos más de una vez para poder encontrar un tipo de solución factible para el problema propuesto. Por esta razón, la mayoría de soluciones son NP-Difíciles. En ocasiones, los algoritmos pueden ser adaptados o limitados bajo ciertas suposiciones, esto permite que se puedan presuponer ciertos patrones que ayuden a encontrar una resolución más rápida y eficaz.

Los problemas NP-Difíciles establecen la complejidad teórica de un problema. En este caso, quiere decir que para encontrar una solución óptima es muy costoso, tanto de recursos computacionales, cómo en el tiempo que toma encontrar la solución.

Las implementaciones suelen estar más centradas en utilizar un enfoque que descomponga el problema en problemas muy específicos, y se busque una solución a estos problemas descompuestos. Algunos pueden adaptar búsquedas heurísticas y otras técnicas como optimización combinatoria, modelado matemático o algoritmos de aproximación.

8.2 Problemas de satisfacción de restricciones (CSP)

El problema de satisfacción de restricciones ha introducido nuevos puntos de vista en múltiples campos de investigación. La principal finalidad de utilizar este tipo de resolución, es encontrar una solución factible para un conjunto de variables sujetas a ciertas restricciones. Hay muchas aplicaciones interesantes dónde se puede aplicar este tipo de problema, como por ejemplo Sudokus o colorear un mapa sin que se repitan colores adyacentes. Sin embargo, intentar resolver estos problemas con fuerza bruta no es una opción, son problemas con muchas posibilidades y has de utilizar de un poder computacional enorme dependiendo de la magnitud del problema. Para poder resolver estos tipos de problemas con una metodología eficiente, se utilizan diferentes tipos de algoritmos y técnicas, que permiten reducir mucho el coste computacional al investigar ciertos patrones, que aseguran que el programa mantenga un estado válido. Por ejemplo, hay algoritmos que aprenden a cada paso y realizan decisiones arriesgadas, que tienen un gran beneficio si resultan ser correctas, y no representan un mayor coste en caso de que no lo sean.

En este proyecto, se pueden utilizar técnicas cómo la propagación de restricciones y algoritmos de búsqueda heurística para encontrar una solución óptima. Si se quisiera utilizar esta metodología, se ha centrar el desarrollo en un método de resolución como el de programación Constraint Programming, junto a un algoritmo booleano cómo SAT para determinar la validez de las restricciones impuestas y encontrar una planificación óptima.

8.3 Definiciones

Búsqueda heurística

El método de búsqueda es utilizado para encontrar posibles soluciones de un problema, pero sin ofrecer ninguna garantía de que existe dicha solución. Se utilizan este tipo de búsquedas, cuando hay demasiados datos como para explorarlos en su totalidad. Con este método se evalúa los datos a medida que se analizan y se realizan predicciones para guiar la búsqueda hacia las soluciones más prometedoras. Hay diferentes tipos de algoritmos que realizan implementaciones específicas sobre diferentes tipos de problemas más comunes.

Modelado matemático

El modelado matemático es un proceso que permite representar diversos tipos de problemas en una representación matemática. Permite definir el problema, identificar las variables necesarias, representar relaciones entre conceptos, con la finalidad de dar una representación que facilite la comprensión y resolución de todo tipo de problemas. Un ejemplo aplicado a la planificación se puede encontrar en *Optimal Job Scheduling*.

Optimización combinatoria

La optimización combinatoria nos permite encontrar la mejor solución dentro de un conjunto finito de soluciones posibles. Este tipo de optimización, utiliza aproximaciones heurísticas que permiten una mayor eficiencia al realizar comprobaciones sobre un conjunto de soluciones extremadamente grande. Este método puede abordar muchos tipos de problemas utilizando diferentes técnicas con diferentes tipos de algoritmos. Dependiendo del problema específico y de las características del mismo, se puede buscar un método de resolución específico, lo que permite adaptarse a cada problema de manera específica y ayudar con una resolución más eficiente.

Constraint programming

Constraint programming es una metodología de programación que se enfoca en programar las restricciones necesarias del programa, y en caso de ser necesario ofrecer una posible solución. Esta metodología es muy utilizada en el ámbito de resolución de problemas, ya que requieren encontrar todas las soluciones posibles utilizando como base unos requisitos. Con esta metodología, se pueden establecer unas limitaciones que han de tener los datos introducidos al programa, y el programa ha de tratar de encontrar una solución válida para el conjunto de datos de entrada junto a las restricciones establecidas. En caso de encontrar un dato que no cumple con un requisito establecido, se puede programar una posible solución que resuelva el conflicto. Hay muchos tipos de empleos de esta metodología y cada uno puede ser muy diferente, hasta puede ser adaptado para cada tipo de problema necesario.

SAT

En los problemas SAT se busca determinar la existencia de una solución para un conjunto de variables y sus restricciones. Se puede realizar un intento de solución de un problema SAT explorando cada opción de cada variable y determinar si algún caso puede ser resuelto. Al ser un problema NP-Completo no hay un algoritmo específico que pueda ayudar a resolver de forma eficiente este tipo de problemas.

CP-SAT

CP-SAT es una técnica de programación que utiliza Constraint Programming para especificar las restricciones, para después poder aplicar diferentes técnicas de resolución de problemas SAT. Los problemas se modelan como en los problemas SAT, con un conjunto de variables, pero además se han podido añadir dominios y restricciones a los datos utilizando CP. Una vez se han podido especificar todos los parámetros necesarios para realizar el problema, antes de intentar la resolución con SAT, se intenta realizar una optimización de los parámetros y los datos de entrada, para reducir las comprobaciones necesarias en la resolución SAT. Al poder especificar tantos tipos de parámetros y ser más eficiente durante la resolución del problema, CP-SAT es utilizado por un amplio espectro de aplicaciones, como la optimización de rutas, diseño de redes, planificación, etc.

9. Prueba de concepto

Se ha realizado el desarrollo básico de unos ejemplos para comprender mejor la viabilidad de la aplicación y la estructura necesaria para su realización. Se ha tenido en cuenta parte de los estudios realizados durante los últimos sprints para realizar dicho desarrollo.

Para las pruebas, simplemente se han establecido unos datos ficticios que simplifican la verificación del resultado del peso de los trabajos. Hay dos tipos de ejemplos, el primero, los trabajos solo se evalúan por su tiempo de duración, en el segundo, su valor depende del tiempo y peso.

El desarrollo más básico de implementar, ha resultado ser el de algoritmos de búsqueda, simplemente se realizan las iteraciones necesarias para obtener el resultado esperado, y después se pueden aplicar restricciones que validen el resultado obtenido para saber si cumple las condiciones necesarias. Sin embargo, cada iteración realizada sobre la colección puede ser muy costosa y hay restricciones que dependen de otras anteriores, dificultando mucho las restricciones entre intervalos de tiempo o posición.

```
thejltres@thejltres-desktop:~/Documents/github/TheJltres/scheduler$ zig run examples/spt.zig
Implementation of the Shortest Processing Time First rule
List of jobs:
Job 0 14
Job 1 67
Job 2 21
Job 3 186
Job 4 211

SPT first sorted:
Job 0 14
Job 2 21
Job 1 67
Job 3 186
Job 4 211
thejltres@thejltres-desktop:~/Documents/github/TheJltres/scheduler$ zig run examples/wspt.zig
Implementation of the Shortest Processing Time First rule
List of jobs:
Job 0 28 124
Job 1 86 171
Job 2 231 228
Job 3 19 9
Job 4 154 28

WSPT first sorted:
Job 1 86 171
Job 0 28 124
Job 2 231 228
Job 3 19 9
Job 4 154 28
```

El segundo ejemplo, se ha realizado utilizando la técnica de desarrollo de satisfacción de restricciones. Esta implementación no implementa todas las características necesarias para realizar aprendizaje booleano o métodos más avanzados que se utilizan en estos casos. Sin embargo, se puede apreciar que es mucho más fácil implementar restricciones en cada iteración de tiempo y se tiene un gran manejo sobre los datos y su validez. El único inconveniente es la dificultad de su implementación y la comprensión del comportamiento del

programa. Pero creo que con una mejor estructuración y una mayor comprensión del problema, se puede mejorar mucho el resultado.

Creo firmemente que utilizar el método de satisfacción de restricciones es la mejor opción para este proyecto. Se obtiene un resultado válido, y se pueden programar todas las posibles restricciones necesarias de manera modular, sin necesidad de realizar iteraciones que pueden realizar múltiples operaciones sobre la matriz original.

```
thejltres@thejltres-desktop:~/Documents/github/TheJltres/scheduler$ zig run examples/csp.zig
Implementation using constraint programming
List of jobs:
Job 0 20
Job 1 59
Job 2 33
Job 3 56
Job 4 62
Searching solution... 255 time slots:
Solution found:
Job 0 20 0
Job 2 33 20
Job 3 56 53
Job 1 59 109
Job 4 62 168
thejltres@thejltres-desktop:~/Documents/github/TheJltres/scheduler$ zig run examples/wcsp.zig
Implementation using constraint programming
List of jobs:
Job 0 41 167
Job 1 68 204
Job 2 35 181
Job 3 15 41
Job 4 89 141
Searching solution... 255 time slots:
Solution found:
Job 2 35 181 0
Job 4 89 141 35
Job 0 41 167 124
Job 1 68 204 165
Job 3 15 41 233
thejltres@thejltres-desktop:~/Documents/github/TheJltres/scheduler$ zig run examples/csp.zig
Implementation using constraint programming
List of jobs:
Job 0 81
Job 1 11
Job 2 64
Job 3 47
Job 4 96
Searching solution... 255 time slots:
No solution was found. Total job time is 299
```

10. Herramientas

Realizar desde el principio una implementación de este calibre, teniendo en cuenta el tiempo definido y la calidad necesaria que requiere el producto final, puede ser un desafío considerable. Por esta razón, se ha realizado una búsqueda de alguna herramienta que ayude a reducir considerablemente la dificultad de la implementación, y asegure una funcionalidad eficiente. Para poder escoger el tipo de solución más acertado para nuestro proyecto, se han tenido en cuenta las siguientes herramientas, y se han comparado sus beneficios y características. Se pretende escoger el método más eficiente que cumpla con las necesidades.

Google OR-TOOLS

OR-TOOLS ofrece múltiples algoritmos de optimización que se pueden adaptar fácilmente a las necesidades específicas de los planificadores. Cuenta con el respaldo de Google, está bien mantenida, y dispone de una API robusta y bien documentada. Con estas características, se puede realizar un desarrollo rápido y fiable. Además, es gratuita y de código abierto con una licencia permisiva. Por defecto, viene con soporte para los lenguajes C++, Python, C# y Java. Desde mi punto de vista parece ser la opción más viable y la que más se ajusta al objetivo de esta aplicación.

IBM ILOG CPLEX Optimizer

CPLEX es una herramienta de pago que pertenece a IBM. Al aceptar las condiciones de uso y comprar la licencia, permite acceder a un conjunto de clases que ayudan a realizar la planificación, además permite acceder a un editor basado en Eclipse que muestra el resultado de las planificaciones en tiempo real. Sin embargo, en este proyecto, no se contempla realizar la suscripción mensual/anual a ninguna herramienta utilizada en el proceso.

Gurobi Optimizer

Gurobi es una empresa que dispone de herramientas y algoritmos para compañías que ayudan a resolver diferentes tipos de problemas. Sin embargo, solo se puede utilizar en entornos de producción si se adquiere la licencia de pago. Esta herramienta viene con soporte para diferentes tipos de lenguaje cómo C, C++, Java, .NET y Python.

OptaPlanner

OptaPlanner es una herramienta escrita en Java que permite optimizar, planear y planificar con restricciones. Aclama ser más sencilla de utilizar que sus competidores, gracias a una gran variedad de características específicamente diseñadas para implementar problemas comunes entre los diferentes tipos de planificación. Combina varios algoritmos avanzados de optimización que permiten encontrar soluciones de alta calidad en tiempos razonables. Esta herramienta ofrece muchos tipos de soluciones para diferentes tipos de problemas empleando enfoques específicos. Esta herramienta es gratuita, de código abierto, y con una licencia permisiva. Los lenguajes de programación con los que se puede utilizar esta herramienta son Java, Kotlin, Scala y con soporte experimental en el momento Python. Sin embargo, para esta aplicación estoy pensando en utilizar un lenguaje de programación de bajo nivel, por ello voy a dejar esta opción como una alternativa a Google OR-TOOLS en caso de que la implementación tenga más dificultad de la esperada.

Implementación

11. Análisis

11.1 Requisitos

Antes de empezar el desarrollo de este proyecto, se van a definir los casos exactos que se busca resolver y qué tipo de soluciones se han pensado para cada problema.

Nuestra principal entrada de datos va a estar compuesta de varios ficheros previamente generados por otra aplicación, los cuales contienen toda la información sobre los objetos a observar y la configuración a aplicar del telescopio.

Para los objetos a observar, el fichero ha de contener los siguientes datos, identificador del objeto, sus coordenadas, prioridad y tiempo de observación.

Para el telescopio, los datos principales que necesitamos son las coordenadas y la altitud, después se han de poder establecer una serie de límites dinámicos que pueden variar según las prestaciones de cada telescopio.

Para el programa, una de las primeras tareas a realizar, es dar formato al documento y guardar todos estos input en memoria y procesar los datos. Una vez se tengan todos los datos introducidos, se ha de realizar la planificación básica. A medida que el proyecto progrese, se van a ir añadiendo utilidades que permitan mejorar la planificación. Por ejemplo, la observación de un objeto puede querer ser obviada, si está detrás de la Luna o muy cerca de ella, ya que la luna puede solapar completamente dicho objeto, o la luna, al ser tan brillante, no dejar ver el objeto a observar. También es poco probable, querer observar un objeto cerca del horizonte, dada la refracción atmosférica. Además sería imposible ver un objeto que esté detrás de la curvatura de la Tierra, es decir, no son visibles desde nuestra posición en la superficie. Hay más restricciones y mejoras que se pueden implementar, para facilitar al observador el estado de los objetos a observar y de ayudar a la planificación, por ello se ha adoptado un estilo de desarrollo iterativo. A medida que se realice la implementación, se irá estudiando el tiempo disponible y que mejoras se puedan implementar. De momento se quiere implementar una base que en un futuro dé cabida a todo este tipo de mejoras de manera modular, sin necesidad de tener que expandir el repositorio principal.

11.2 Funcionalidades descartadas

Dadas las limitaciones de recursos en este proyecto, no se van a poder realizar todas las tareas esperadas para este proyecto, las que no se puedan realizar, van a quedar documentadas aquí, junto a su razón y el estado en que están en caso de que se haya realizado parte de su implementación.

Múltiples telescopios

Actualmente, en el proyecto del TFRM no hay planes de incorporación de un nuevo telescopio o el uso en sincronía con otros telescopios. Es una implementación que no es requerida, y conlleva mucho trabajo de implementar. Si se ha de planificar en múltiples telescopios, has de tener en cuenta la localización y las características de cada uno para poder realizar planificaciones ajustadas a cada configuración. Sin embargo, en la configuración se deja abierta la posibilidad. En caso de que a un tercero le interese la opción, sólo tendrá que realizar la implementación de los cálculos con las propiedades específicas del telescopio y no se tendrá que cambiar toda la estructura del código.

Interfaz gráfica

Este proyecto está pensado para usuarios avanzados y no se requiere una interfaz. Además, este proyecto está pensado para poder ser utilizado en servidores. Los usuarios han de ser capaces de poder introducir un comando con varios parámetros para poder realizar las planificaciones. No entra dentro del proyecto realizar una configuración visual o desde un entorno web en estos momentos. Al ser una funcionalidad opcional que no conlleva ningún cambio sustancial para el usuario final, no se planea realizar esta funcionalidad.

Base de datos

Una base de datos es la mejor manera de tener organizados todos los datos con los que trabajamos. Sin embargo, por el momento es opcional y no requerido. Los usuarios finales están satisfechos con utilizar ficheros para llevar la planificación. Actualmente, hay un programa externo que genera los ficheros con los datos de los objetos, y la configuración del telescopio es estática y solo se realizan cambios de manera puntual. Por esa razón, en caso de implementar esta funcionalidad, se ha de crear un sistema de importación de los datos actuales a la base de datos. Además, se necesita realizar la configuración del servidor con la base de datos y añadir una dependencia al proyecto para que el cliente comunique las interacciones con la base de datos. Creo que es una funcionalidad que requiere de mucho tiempo para ser utilizada y no merece la pena por el momento.

Validar posiciones

No se espera realizar validaciones sobre posiciones de objetos o puntos. Si las coordenadas introducidas no son correctas, el resultado puede no ser el esperado. Es trabajo del observador validar que las coordenadas introducidas al programa son válidas para la noche de observación.

11.3 Funcionalidades opcionales

Hay algunas funcionalidades que pueden ser útiles, pero que quedan fuera del alcance del proyecto. Las más importantes estarán documentadas aquí y si el tiempo del proyecto lo permite, se realizará su implementación.

Características del telescopio

El telescopio TFRM tiene muchas características específicas que cambian las posibilidades de la planificación. Se tiene pensado introducir todas las restricciones posibles que de tiempo a programar, pero no se espera realizar la implementación de todas las restricciones que puede tener el telescopio.

Intervalos de observación específicos

Normalmente, no se puede observar durante la totalidad del día, se realizan observaciones de noche, una vez el sol ya se ha puesto, entre los crepúsculos astronómicos. Hay veces que puede haber meteorología adversa prevista y no se desea arriesgar el material y se realizan observaciones entre intervalos seguros. Todas estas características no están previstas de implementar por el momento. Sin embargo, en el futuro, espero que si se consigue una aproximación modular, se pueden implementar sin problema, incluso con casos adversos.

Aproximación modular

La aproximación modular es una funcionalidad que espero poder implementar si las pruebas me lo permiten. No tengo mucho conocimiento con la herramienta de planificación, y aún no creo que pueda realizar esta funcionalidad. Sin embargo, es muy probable, que dada su importancia, sea de las primeras a realizar si realizan las funcionalidades principales.

11.4 Descripción de datos

Telescopio

La principal información necesaria, es conocer la localización del telescopio, dónde está ubicado en la Tierra. Vamos a necesitar la Latitud, la Longitud y la Altitud (en metros) del telescopio para poder conocer nuestra posición y poder establecer la dirección a los objetivos.

Hace falta definir un rango de horas para la observación, no se obtienen datos durante el día, ya que el sol brilla mucho, por ello esperamos a que se haga de noche, y se espera a una cierta hora, entre los crepúsculos astronómicos. Se va a definir una hora de inicio del crepúsculo y otra de finalización en ángulos.

El telescopio tiene unos límites de observación, algunos de movimientos físicos, y otros de visibilidad. Se va a necesitar un límite vertical de observación, porque si un objeto está muy al límite del horizonte, hay mucha refracción del aire y el objeto que se observa no va a estar muy nítido. Vamos a tener un límite verticalidad para la observación en grados. Otro parámetro que se va a necesitar para poder observar objetos nítidos, es conocer la posición de la Luna, y establecer una distancia mínima de diferencia con el objeto observado. Si un objeto está muy cerca de la Luna, el brillo de la Luna puede ocultar el objeto a observar. La diferencia de distancia entre los dos objetos se define en grados. También, dada la limitación física de la rotación del telescopio, solo podemos observar dentro de una horquilla de ángulos. Se va a necesitar un ángulo máximo de observación hacia el Norte y otro ángulo de observación hacia el Sud.

Otro valor que se aplica a todos los objetos a observar, puede ser las horas de observación desde su culminación. Puede ser interesante, solo observar un objeto en un rango de tiempo determinado, con su culminación en el centro. De esta manera, se puede asegurar que la visibilidad del objeto será la mejor posible.

Objetivos

Para conocer los objetivos que vamos a necesitar observar, se necesitan pocos datos, ya que la mayoría, se pueden obtener con cálculos matemáticos y no es necesario guardarlos.

Vamos a guardar el identificador del objeto, y su posición. El identificador va a ser un número único para cada objeto. La posición del objeto, se va a guardar como coordenadas del cielo, utilizando Ascensión Recta (RA) y Declinación (DEC). Estas coordenadas son diferentes a las de la tierra, ya que no dependen de la posición en la rotación de la tierra, son las Coordenadas Celestes. Para poder obtener una Coordenada Celeste, se necesita pensar en la Tierra cómo una Esfera Celeste. Entonces, desde el Ecuador Celeste, se ha establecido un punto de origen común, con el que se puede obtener una

dirección de observación, este punto es el Punto Aries. El punto aries es el punto dónde el Sol cruza del hemisferio celeste Sur al hemisferio celeste Norte. Desde este punto de origen, la RA, es una distancia espesada en ángulos, que recorre el Ecuador Celeste. Se mide desde el Punto Aries, en dirección al Este. La DEC, es el ángulo que forma el objeto con el Ecuador Celeste. También necesitamos una prioridad para cada objeto y el tiempo necesario de observación de cada objeto.

11.5 Historias de usuario

Importar configuración telescopio

Título:	Como usuario quiero poder importar la configuración correcta para utilizar el telescopio.
Descripción:	Como usuario, necesito poder establecer una configuración para el telescopio, que permite especificar los parámetros de observación desde un fichero. Por lo tanto, necesito una opción que me permita incluir el archivo de configuración.
Criterios de aceptación:	1. Como usuario, debo poder utilizar el parámetro '--telescope' junto con el ejecutable de la aplicación para cargar un fichero con la configuración. 2. Como usuario también debo poder utilizar el parámetro reducido '-t'. 3. El parámetro '-t' ha de realizar la carga de la configuración en el ejecutable. 4. El ejecutable debe proporcionar resultados claros y legibles que muestren el estado de la planificación.

Importar objetos

Título:	Como usuario quiero poder importar los objetos que se pueden planificar durante una noche.
Descripción:	Como usuario, necesito importar todos los objetos que se van a observar durante la noche para realizar la planificación. Los objetos van a estar en un fichero, por lo tanto, necesito que me permita incluir el archivo con los objetos.
Criterios de aceptación:	1. Como usuario, debo poder utilizar el parámetro '--import-objects' junto con el ejecutable de la aplicación para cargar un fichero con los objetos. 2. Como usuario también debo poder utilizar el parámetro reducido '-i'. 3. El parámetro '-i' ha de realizar la carga de los objetos a observar en el ejecutable. 4. El ejecutable debe proporcionar resultados claros y legibles que muestren el estado de la planificación.

Fecha de observación

Título:	Como usuario quiero poder establecer la fecha de observación de los objetos.
Descripción:	Como usuario, necesito poder establecer la fecha sobre la que el trabajador va a realizar las planificaciones. La fecha será la fecha en la que se empiezan a realizar las observaciones.
Criterios de aceptación:	1. Como usuario, debo poder utilizar el parámetro '--date' junto con el ejecutable de la aplicación para establecer la fecha de la observación. 2. El ejecutable debe proporcionar resultados claros y legibles que muestren el estado de la planificación.

Mostrar versión

Título:	Como usuario quiero poder conocer la versión de mi ejecutable.
Descripción:	Como usuario, necesito poder conocer la versión de mi programa para asegurar la compatibilidad y acceso a las últimas funcionalidades y correcciones.
Criterios de aceptación:	1. Como usuario, debo poder utilizar el parámetro '--version' junto con el ejecutable de la aplicación para mostrar la versión del ejecutable. 2. Como usuario, también debo poder utilizar el parámetro reducido '-v' 3. El parámetro '-v' ha de poder mostrar la versión actual del ejecutable y sus dependencias.

Mostrar ayuda

Título:	Como usuario quiero poder conocer los parámetros que se pueden utilizar.
Descripción:	Como usuario, necesito poder conocer todos los parámetros que se pueden utilizar. Necesito poder saber todas las funcionalidades que me ofrece el software.
Criterios de aceptación:	1. Como usuario, debo poder utilizar el parámetro '--help' junto con el ejecutable de la aplicación para mostrar la ayuda del ejecutable. 2. Como usuario, también debo poder utilizar el parámetro reducido '-h' 3. El parámetro '-h' ha de poder mostrar los parámetros disponibles para el usuario 4. El parámetro '-h' ha de poder mostrar una breve información del uso de cada parámetro. 5. Como usuario, debo poder observar la versión del ejecutable.

Mostrar registros

Título:	Como usuario quiero poder saber qué pasos ha realizado el programa.
Descripción:	Como usuario, necesito saber qué pasos ha realizado el programa para asegurarse de que está funcionando correctamente o realizar alguna modificación
Criterios de aceptación:	1. Como usuario, debo poder utilizar el parámetro '--verbose' junto con el ejecutable de la aplicación para mostrar la versión del ejecutable. 2. El parámetro '--verbose' ha de poder mostrar todos los registros del programa

11.6 Casos de uso

Importar configuración

Descripción:	Mostrar cómo se va a realizar la importación del fichero de configuración.
Actores:	Usuario
Precondiciones:	• El usuario debe tener acceso a una terminal • Un fichero con la configuración del telescopio ha de existir • El fichero de configuración del telescopio ha de tener el formato correcto
Flujo:	1. El usuario inicia la terminal 2. El usuario escribe el comando del ejecutable seguido del parámetro '-t' o '--telescope' 3. El usuario introduce el nombre del telescopio o el fichero a importar. 4. El usuario puede introducir otros parámetros. 5. El usuario puede revisar los resultados de la planificación.
Postcondiciones:	• Si no se introduce un valor para el parámetro, se mostrará un error. • El usuario puede introducir otros parámetros. • El usuario puede revisar los resultados de la planificación. • El usuario puede volver a repetir la planificación.
Errores:	• Si no se introduce un valor para el parámetro, se mostrará un error. • Si el fichero de configuración del telescopio no

	tiene un formato correcto, se mostrará un error. • Si la configuración del telescopio no tiene un formato válido, se mostrará un mensaje indicando el error. • Si no se puede realizar la planificación, se mostrará un mensaje indicando el error.
--	---

Importar objetos

Descripción:	Mostrar cómo se va a realizar la importación de los objetos a observar.
Actores:	Usuario
Precondiciones:	• El usuario debe tener acceso a una terminal • Un fichero con los objetos a importar ha de existir • El fichero de objetos a observar ha de tener el formato correcto
Flujo:	1. El usuario inicia la terminal 2. El usuario escribe el comando del ejecutable seguido del parámetro '-i' o '--import-objects' 3. El usuario introduce un fichero con los objetos a observar. 4. El usuario puede introducir otros parámetros. 5. El usuario puede revisar los resultados de la planificación.
Postcondiciones:	• Si no se introduce un valor para el parámetro, se mostrará un error. • El usuario puede introducir otros parámetros. • El usuario puede revisar los resultados de la planificación. • El usuario puede volver a repetir la planificación.
Errores:	• Si el fichero de objetos a observar no tiene un formato correcto, se mostrará un error. • Si el fichero de objetos no tiene un formato válido, se mostrará un mensaje indicando el error. • Si no se puede realizar la planificación, se mostrará un mensaje indicando el error.

Fecha de observación

Descripción:	Mostrar cómo se va a realizar la importación de la fecha de observación.
Actores:	Usuario
Precondiciones:	• El usuario debe tener acceso a una terminal • El formato de la fecha ha de ser dd/MM/yyyy
Flujo:	1. El usuario inicia la terminal 2. El usuario escribe el comando del ejecutable seguido del parámetro '--date' 3. El usuario introduce la fecha con el formato correcto 4. El usuario puede introducir otros parámetros. 5. El usuario puede revisar los resultados de la planificación.
Postcondiciones:	• El usuario puede introducir otros parámetros. • El usuario puede revisar los resultados de la planificación. • El usuario puede volver a repetir la planificación.
Errores:	• Si no se introduce un valor para el parámetro, se mostrará un error. • Si la fecha no tiene el formato correcto, se mostrará un error y el formato esperado. • Si la fecha no es correcta se mostrará un error • Si no se puede realizar la planificación, se mostrará un mensaje indicando el error.

Mostrar versión

Descripción:	Mostrar cómo se va a realizar la acción de visualización de la versión del software.
Actores:	Usuario
Precondiciones:	• El usuario debe tener acceso a una terminal
Flujo:	1. El usuario inicia la terminal 2. El usuario escribe el comando del ejecutable seguido del parámetro '-v' o '--version'
Postcondiciones:	• Si el ejecutable recibe más parámetros se ignorarán • El usuario puede ver la versión del programa • El usuario puede volver a ejecutar el software.
Errores:	No se espera poder encontrar errores en este caso.

Mostrar ayuda

Descripción:	Mostrar cómo se va a realizar la acción de visualización de la ayuda del software.
Actores:	Usuario
Precondiciones:	• El usuario debe tener acceso a una terminal
Flujo:	1. El usuario inicia la terminal 2. El usuario escribe el comando del ejecutable seguido del parámetro '-h' o '--help'
Postcondiciones:	• Si el ejecutable recibe más parámetros se ignorarán • El usuario puede ver la ayuda de todos los comandos y su uso • El usuario puede volver a ejecutar el software. • El usuario ha de poder observar la versión del ejecutable
Errores:	No se espera poder encontrar errores en este caso.

Mostrar registros

Descripción:	Mostrar cómo se va a realizar la acción de mostrar los registros de ejecución del programa.
Actores:	Usuario
Precondiciones:	• El usuario debe tener acceso a una terminal
Flujo:	1. El usuario inicia la terminal 2. El usuario introduce todos los comandos para realizar la planificación. 3. El usuario escribe el comando del ejecutable seguido del parámetro '-v' o '--verbose'
Postcondiciones:	• El usuario puede saber todos los pasos que está realizando el planificador • El usuario puede revisar los resultados de la planificación. • El usuario puede volver a repetir la planificación.
Errores:	No se espera poder encontrar errores en este caso.

Planificación básica

Descripción:	Mostrar cómo se va a realizar la acción de planificación
Actores:	Usuario
Precondiciones:	• El usuario debe tener acceso a una terminal • El usuario introduce un fichero de configuración de telescopio válida. • El usuario introduce un fichero de objetos a planificar válidos.
Flujo:	1. El usuario inicia la terminal 2. El usuario introduce todos los comandos para realizar una planificación válida.
Postcondiciones:	• El usuario puede revisar los resultados de la planificación. • El usuario puede volver a repetir la planificación.
Errores:	• Si los parámetros no son válidos, se mostrará el error correspondiente. • Si no se puede realizar la planificación, se mostrará un mensaje indicando el error.

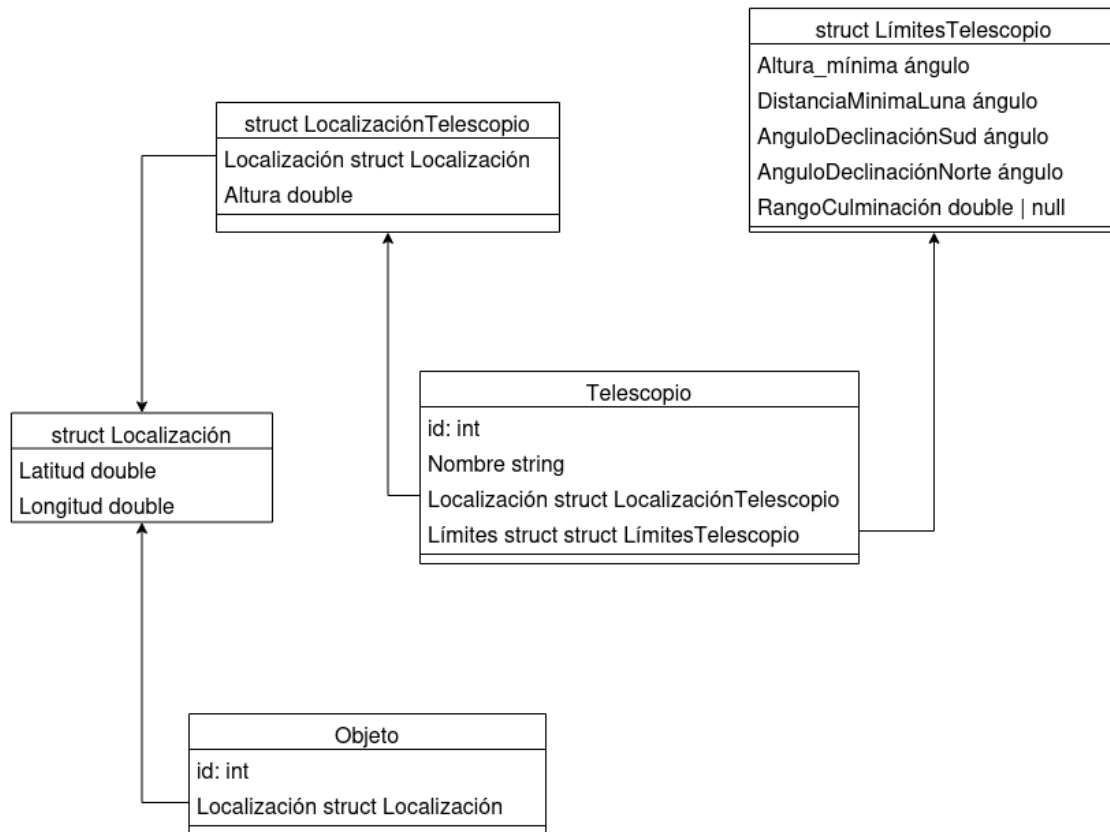
11.7 Planes de Test

El desarrollo de una aplicación enfocada a fines científicos, en este caso a la observación de objetos en el espacio, requiere una implementación de calidad y con la máxima fiabilidad posible. Es fundamental implementar un plan de pruebas exhaustivo que garantice un sistema robusto, y que permita verificar una correcta funcionalidad en todos los posibles casos. Se va a realizar un conjunto de pruebas que permita validar el funcionamiento de todos los aspectos lógicos que tiene la implementación. No se van a realizar pruebas unitarias del código de las librerías externas al proyecto, solo de los aspectos lógicos desarrollados en la aplicación.

Para llevar a cabo esta tarea, se ha tenido en cuenta la librería CPlusPlusUnit. Esta librería, más comúnmente conocida como CPPUnit, proporciona un conjunto de clases y macros que permiten definir y ejecutar pruebas unitarias de manera sencilla. Es muy sencilla en comparación a otras librerías más grandes, pero dado el tamaño de la aplicación que se va a crear, va a permitir un desarrollo más eficiente sin afectar a la seguridad de la calidad del código.

12. Diseño

12.1 Diagrama de clases



Telescopio

Id: Valor numérico único que identifica el Telescopio.

Nombre: Nombre para identificar el Telescopio de manera más sencilla para el usuario.

Localización: Valores con la posición del Telescopio.

Objeto

Id: Valor número único para identificar el Objeto.

Localización: Localización con Coordenadas Celestes del Objeto.

Localización

Latitud: Valor decimal que identifica el valor asociado a la Latitud de la clase asociada.

Longitud: Valor decimal que identifica el valor asociado a la Longitud de la clase asociada.

Localización Telescopio

Localización: Coordenadas geográficas del Telescopio.

Altura: Valor decimal con los metros de la altura del Telescopio.

Límites Telescopio

Altura mínima: Ángulo de altura mínimo del Objeto a observar.

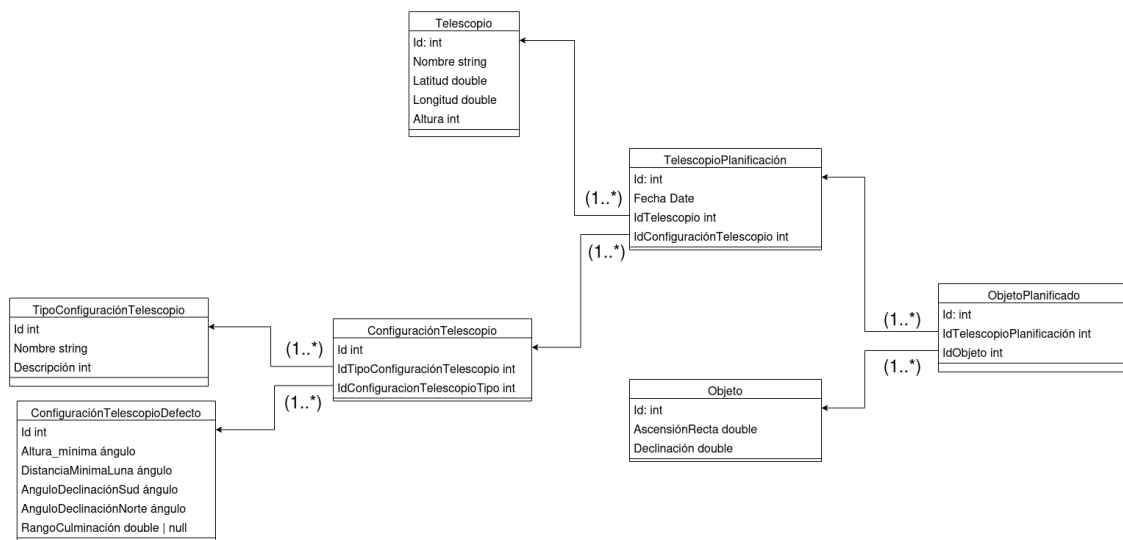
Distancia mínima luna: Ángulo mínimo del Objeto observado en relación con la Luna.

Ángulo de declinación Sud: Ángulo mínimo del Objeto a observar en dirección Sud.

Ángulo de declinación Norte: Ángulo mínimo del Objeto a observar en dirección Norte.

Rango de Culminación: Tiempo posible de observación del Objeto antes y después de culminar.

12.2 Modelo entidad-relación



Esta imagen es una representación conceptual que muestra los campos necesarios para guardar las observaciones realizadas a lo largo del tiempo en una base de datos. Esta solución es un posible objetivo a largo plazo, que permite guardar en una base de datos los resultados de las planificaciones.

Telescopio

Entidad que representa los telescopios que se utilizan para realizar las planificaciones.

Atributos:

- Id (clave primaria)
- Nombre: Nombre para mayor facilidad de identificación del telescopio
- Latitud: Latitud de la posición del telescopio
- Longitud: Longitud de la posición del telescopio
- Altura: Altura en metros de la posición del telescopio

Objeto

Entidad que representa los objetos que se utilizan para realizar diferentes planificaciones. Al ser objetos muy estáticos, no se necesita tener guardado su movimiento a lo largo del tiempo.

Atributos:

- Id (clave primaria)
- Ascensión Recta: Valor de la ascensión recta de un objeto
- Declinación: Valor de declinación de un objeto

Tipo Configuración Telescopio

Entidad que representa la manera que se han de aplicar diferentes configuraciones de telescopio.

Atributos:

- Id (clave primaria)
- Nombre: Nombre de la configuración, para mayor facilidad del usuario
- Descripción: Permite describir los mejores casos para aplicar esta configuración

Configuración Telescopio [Defecto]

Entidad dinámica que permite definir los parámetros de configuración necesarios para realizar configuraciones.

Atributos:

- Id (clave primaria)
- Los demás atributos son dinámicos y pueden variar. Se ajustan al tipo de configuración necesaria para el operador.

Configuración Telescopio

Entidad que representa una configuración de limitaciones para utilizarse en las planificaciones.

Atributos:

- Id (clave primaria)
- IdTipoConfiguraciónTelescopio: Tipo de configuración a aplicar para el telescopio.
- IdConfiguraciónTelescopio [Defecto]: Atributos específicos para el tipo de configuración del telescopio escogida.

Telescopio Planificación

Entidad que representa las planificaciones realizadas a lo largo del tiempo. Da a conocer la fecha de la observación, el telescopio utilizado y la configuración de las limitaciones de la observación.

Atributos:

- Id (clave primaria)
- Fecha: Fecha de observación
- IdTelescopio: Identificador del telescopio utilizado para la observación.
- IdConfiguraciónTelescopio: Identificador de la configuración de las limitaciones utilizadas para la observación.

Objeto Planificado

Entidad que representa el resultado de la planificación para los objetos en una fecha concreta con una configuración determinada.

Atributos:

- Id (clave primaria)
- IdTelescopioPlanificación: Identificador de la planificación asignada al objeto.
- IdObjeto: Identificador del objeto asignado a planificar.

13. Arquitectura

13.1 DevOps

Control de versiones

Los archivos relacionados con la aplicación, como el código fuente y la documentación, se almacenan en un repositorio Git. De esta manera se permite a los desarrolladores colaborar y realizar un seguimiento de los cambios efectuados a lo largo del tiempo.

Integración Continua (CI)

Se han configurado pipelines automatizadas en GitHub para verificar la funcionalidad de la aplicación a través de las pruebas unitarias implementadas. Se pueden realizar expansiones en el futuro para integrar pruebas de integración u otros tipos de validaciones.

13.2 Gestión de la configuración

La configuración de esta aplicación se aplica exclusivamente introduciendo parámetros en la línea de comandos al realizar la planificación. Hay algunos parámetros que permiten modificar el comportamiento de la aplicación, por ejemplo, mostrar información sobre la ejecución. Al utilizar solo parámetros, se facilita mucho la implementación y simplifica mucho para el usuario final el uso de la aplicación.

13.3 Trazabilidad

La configuración de trazabilidad de la aplicación se activa mediante el uso de un parámetro al ejecutar la aplicación. Al activar dicha función, el programa muestra toda la información sobre la ejecución del programa, indicando qué métodos están siendo utilizados, qué pasos se están siguiendo y mucha más información que puede ser útil para identificar posibles errores o problemas. Al activar esta característica, se facilita el proceso de depuración y se puede obtener una mejor información sobre el comportamiento del programa. Incluso el usuario final, puede hacer reportes directamente y adjuntar el resultado de la trazabilidad para una mayor información sobre los errores.

13.4 Modularidad

Para conseguir un enfoque modular para aplicar restricciones, es necesario establecer desde el principio un buen sistema de abstracción que permita extender las posibilidades del programa. En este caso, el usuario ha de poder añadir, quitar o modificar restricciones de manera sencilla sin necesidad de tocar código fuente de la lógica principal de la aplicación. Creo que es mucho más simple tener que modificar código que está enfocado en una sola funcionalidad, y puede permitir a los usuarios finales un mayor abanico de opciones para realizar las observaciones.

13.5 Testing

Antes de poder entregar la aplicación en su estado final, se ha de estar seguro de su correcto funcionamiento. Por esa razón, antes de poder realizar cualquier lanzamiento, se ha de asegurar que los test y el CI ha sido ejecutado y ha tenido un resultado exitoso. Se han de realizar pruebas exhaustivas e ir añadiendo más en función de la información que nos brinde el usuario final. Se pretende cumplir con unos estándares de calidad y satisfacción para el usuario final.

14. Gestión

14.1 Estudio de mercado

Con este estudio de mercado, se pretende conseguir el conocimiento del estado actual de las aplicaciones que ofrezcan un servicio similar al de este proyecto. Además, si alguna herramienta compartiera su procedimiento para realizar las programaciones de forma abierta, también se podría investigar para estudiar el funcionamiento en profundidad de una herramienta ya creada. Una vez ya se haya obtenido la información necesaria para poder plantear nuestro proyecto, se pretende realizar un análisis retrospectivo que nos permita mejorar las prestaciones de este proyecto.

Alternativas

He encontrado pocas aplicaciones que realicen planificaciones enfocadas para ayudar a los astrónomos, la mayoría se centran en planificar todas las tareas a realizar de uno o varios observatorios. Por ejemplo:

He encontrado un proyecto de código abierto, dónde te permite instalar en tu propio servidor todo el software necesario para la integración de todas las funcionalidades que incluye su aplicación. Esta aplicación se llama Observatory Control System, está basada en microservicios y tiene una gran cantidad de utilidades. Sin embargo, en mis pruebas hay algún fallo en el servicio del planificador y no se puede ejecutar. Los demás componentes parecen ser muy útiles, pero es demasiado complicado y contiene demasiadas utilidades que no se contemplan realizar en este proyecto. Para realizar el planificador, utilizan OR-Tools con soporte para los algoritmos SCIP, CBC, GLPK y Gurobi (con la licencia).

Otra alternativa que he encontrado es Arcsecond. Esta aplicación ofrece muchas utilidades para los observatorios a través de un portal web. Tiene muchas utilidades que ayudan a gestionar los centros de observación, además de un planificador para observar los objetos. Hay una versión gratuita y un video de la demo de dicha versión. No obstante, no hay un planificador cómo el que se está planteando en este proyecto, pero sí que existe un planificador que muestra el tiempo de los observadores y ayuda a realizar la gestión de un centro de observación. Creo que algunas de las ideas mostradas en esta aplicación pueden ser interesantes y de mucha utilidad.

Otras herramientas que he observado, han sido ACP Observatory Control Software y SkyTools. Sin embargo, el código de estas aplicaciones es privado y se requiere una licencia de pago para utilizar su software. Ambas llevan mucho tiempo en el mercado y ofrecen características muy específicas y esenciales para realizar todo tipo de gestión de los observatorios y planificación de las observaciones si adaptas tu trabajo a sus herramientas.

Sin embargo, mi idea de cómo ha de ser implementado este proyecto difiere un poco de todas estas alternativas, la que más se parece a este proyecto, es la primera comentada, Observatory Control System. Pero se intenta resolver el problema con otro tipo de enfoque.

14.2 DAFO

- **Fortalezas**

La mayor fortaleza de este proyecto, es el tamaño reducido del equipo, 1 persona. Además, una idea clara de las funcionalidades básicas que se han de realizar.

- **Debilidades**

Una de las mayores debilidades que tiene el proyecto es la experiencia utilizando algoritmos de optimización. Además, si se alarga el desarrollo del proyecto, con los pocos recursos de tiempo disponible, es muy difícil entregar todas las prestaciones y realizar un mantenimiento futuro

- **Oportunidades**

Este proyecto nace gracias a la necesidad de realizar una implementación específica para el equipo de observación del TFRM. Además, se pueden obtener muchos conocimientos sobre algoritmos de optimización, que pueden resultar útiles en futuros proyectos.

- **Amenazas**

Este proyecto tiene un reducido alcance y tiempo limitado para la implementación. Hay muchas otras herramientas que pueden ofrecer más recursos.



15. Programación

En este apartado se pretende detallar y documentar todos los pasos que se van a realizar a lo largo de la implementación del proyecto. Desde la razón de ciertas decisiones hasta detalles del funcionamiento de las implementaciones más importantes. Se pretende ayudar a comprender los fundamentos y la lógica del funcionamiento de la aplicación.

15.1 Lenguaje de programación

El lenguaje de programación que se va a utilizar para este proyecto, no ha sido tan sencillo de escoger. He estado mucho tiempo pensando en qué lenguaje puede ser el mejor para esta aplicación, y quiero escoger un lenguaje que cumpla con ciertas características. Sin embargo, para utilizar la librería de OR-Tools con los lenguajes de programación soportados, estoy limitado a realizar la implementación con los siguientes lenguajes: C++, Python, Java o .Net.

Python es un lenguaje de programación fácil de utilizar, sin embargo, la eficiencia no es uno de sus puntos fuertes. Aunque en este proyecto, solo se realicen llamadas a la librería de OR-Tools, hay que tratar todos los datos y darles formato antes de poder utilizarlos. Además, los tipos de las variables son dinámicos y requieren de un intérprete para poder funcionar.

Al estar buscando un lenguaje de bajo nivel que permita un gran control sobre todos los aspectos de la aplicación, voy a decantarme por C++. Java o .Net son grandes alternativas, pero requieren de capas de abstracción entre el código y el Sistema Operativo, y además utilizan técnicas para manejar la memoria por su cuenta sin poder obtener el control en todos los aspectos.

Sin embargo, lenguajes como C o C++, han estado bajo el punto de mira de varias instituciones, debido al gran peligro que puede suponer dejar al desarrollador todo el control para manejar la memoria que utiliza el programa y las pocas comprobaciones de la validez del código que realiza el propio lenguaje. Por ello he estado estudiando la existencia de algunas alternativas más modernas que puedan ser utilizadas con OR-Tools. A pesar de todo, no he encontrado una solución viable, OR-Tools está escrito en C++, y utiliza una herramienta llamada SWIG que permite crear traducciones que pueden ser llamadas desde otros lenguajes. Han hecho algunos avances para poder soportar Go, pero está un poco lejos de ser estable. Otra alternativa es crear un servidor utilizando ProtoBuf y poder realizar llamadas desde los otros tipos de lenguaje, pero la implementación seguirá estando escrita en C++ y no creo que valga la pena el tiempo y esfuerzo necesario para realizar toda esta implementación. Así que para la implementación de este proyecto se va a utilizar C++ en su totalidad.

He utilizado Zig para algunos ejemplos mostrados anteriormente, esto se debe a que me es mucho más sencillo de leer, escribir, compilar y entender. La compilación y ejecución del código es mucho más sencilla que con C o C++, dónde se necesitan herramientas de terceros que tienen sus propias dependencias, configuración y asunciones sobre la estructura que ha de tener tu código. Si en un futuro se puede cambiar la compilación de CMake a Zig, se puede buscar una manera de integrar Zig en el proyecto y establecer interoperabilidad sin ningún tipo de problema o contratiempo. Creo que Zig es muy interesante, y es muy importante tener en cuenta sus avances. Aunque no esté en una versión estable, se debe a la falta de funcionalidades que están siendo añadidas, y que la estructura del lenguaje aún puede recibir algunas modificaciones. No obstante, hay ya muchos proyectos en producción que utilizan Zig en su totalidad, y las partes funcionales actuales son más que suficientes para realizar unos ejemplos fiables.

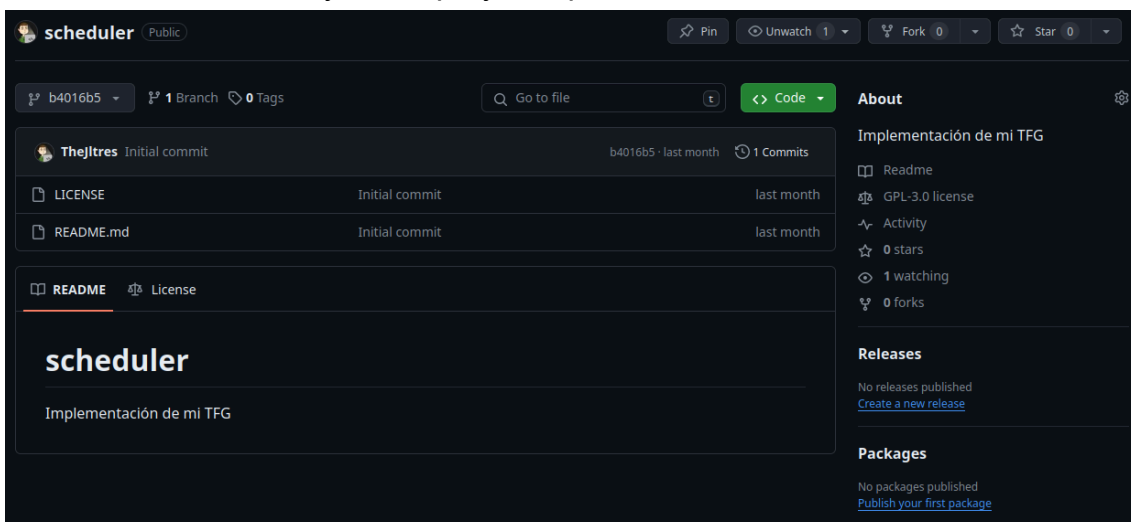
15.2 Estructura del proyecto

Repositorio

Para realizar el control de versiones, vamos a utilizar Git. Git es un sistema de control de versiones que permite tener un historial de todos los cambios realizados a través de commits.

También, vamos a utilizar GitHub, una plataforma online que proporciona herramientas para compartir proyectos y funcionalidades que ayudan a colaborar y gestionar la implementación del proyecto.

Para empezar, se ha creado el repositorio utilizando GitHub. Se ha escogido la licencia GNU GPL 3.0 y es un proyecto público.



Estructura de carpetas

- build/bin
Carpeta final dónde se compilan todos los ejecutables de la aplicación.
- docs/
En este directorio, vamos a guardar un manual sobre cómo se utiliza este ejecutable y otras posibles explicaciones necesarias para comprender partes esenciales del proyecto. También se va a añadir este documento una vez terminado.
- examples/
En este directorio, se guardan todos los ejemplos que se han realizado en este documento. Pretende ayudar al usuario final a comprender los conceptos más complicados con ejemplos simplificados para cada uso.
- plugins/
En esta carpeta el usuario podrá tener todas las extensiones necesarias para realizar la observación. Se pueden añadir o quitar elementos dinámicamente para que el programa los pueda incorporar de manera sencilla.
- src/
Contiene todo el código fuente de la aplicación.
- test/
Contiene todas las pruebas automatizadas que comprueban los aspectos básicos de la aplicación.

15.3 Dependencias

Para realizar la implementación de este proyecto, se pretende utilizar C++, junto con CMake, para realizar la compilación del proyecto y las dependencias asociadas. Las dependencias que se van a utilizar son: OR-Tools, Argh, mINI y libastro.

OR-Tools

OR-Tools va a ser la principal dependencia de este proyecto. Esta librería ofrece muchas funcionalidades enfocadas a la optimización, y es una herramienta muy útil para realizar planificaciones. Destaca por su eficiencia y versatilidad en la resolución de problemas complejos mediante técnicas de programación funcional.

Argh

Esta dependencia es una pequeña librería que facilita el tratado de los parámetros en la línea de comandos. Se pueden definir fácilmente los parámetros necesarios para el ejecutable y luego acceder a esos parámetros

desde el código de manera sencilla. Es una manera sencilla de validar y programar un posible caso para cada posible parámetro introducido. Es de código abierto y solo necesitas un fichero de encabezado para interactuar con sus funcionalidades.

mINI

Para obtener los datos de configuración del telescopio, he escogido utilizar el formato de datos INI. INI es un formato común utilizado para almacenar la configuración de aplicaciones simples en archivos de texto plano, además permite organizar los datos en secciones y pares clave-valor.

mINI es una pequeña librería que simplifica el proceso de tratado de datos de este formato de archivo. mINI permite administrar los datos de los ficheros INI de manera simple. Se pueden validar y obtener los valores requeridos rápidamente y sin mucha dificultad.

libastro

Libastro es una librería utilizada en el proyecto XEphem. Esta librería escrita en C se utiliza para realizar cálculos astronómicos precisos y eficientes. Además, está diseñada de manera portable y es fácil de integrar en el proyecto. Permite determinar la posición de cuerpos celestes frecuentes, como la Luna o planetas del sistema solar en un momento específico en el tiempo. También permite realizar cálculos de distancia entre objetos, algo que nos interesa mucho.

15.4 Implementación producto final

Para empezar el proyecto, hemos creado un fichero de configuración CMake para especificar la forma en la que se ha de compilar el proyecto y las dependencias.

Documentación

He escogido utilizar la utilidad Man para visualizar toda la documentación del ejecutable. Man es una herramienta muy eficaz, que de manera sencilla, permite mostrar manuales de referencia para ejecutables. Sigue un formato consistente y estandarizado. Además, viene preinstalado en muchas distribuciones Unix y Linux por defecto. Es una herramienta muy conocida y utilizada.

Cmake

Cmake es una herramienta de código abierto que es utilizada para automatizar el proceso de compilación de las aplicaciones. Para este proyecto, se puede compilar nuestra aplicación, descargar el código de las dependencias y realizar una vinculación estática.

Para configurar OR-Tools, se ha seguido la guía oficial de Google para instalar las herramientas necesarias de C++, y se ha adaptado el fichero de CMake para poder realizar una compilación automática de todo el proyecto. Se han tenido que realizar algunas modificaciones en las variables globales de C++ que establecen una serie de normas requeridas al utilizar Or-Tools.

La descarga de la dependencia se realiza directamente contra el repositorio de GitHub con la versión v9.9. Argh y mINI se descargan de la misma manera, Argh con la versión v1.3.2 y mINI con la versión 0.9.15. La dependencia libastro, ha tenido que ser incorporada a mano, ya que su encabezado principal no es compatible con C++, y se han tenido que realizar unas modificaciones al ser utilizado desde el proyecto. Está ubicada en la carpeta *include* dentro de *src*. Libastro no realiza modificaciones muy continuadas, sin embargo, puede ser interesante realizar algún fork y automatizar esta tarea de compilación.

Al compilar, se han de ejecutar dos comandos, y se crea el ejecutable *scheduler* en la carpeta *build/bin*.

```
thejltres@thejltres-desktop:~/Documents/github/TheJltres/scheduler$ cmake --build build -j 6 | tail -n +150
[ 98%] Built target scheduler
[ 98%] Built target solve
[ 98%] Built target vector_bin_packing
[ 98%] Built target sat_runner
[100%] Built target flatzinc
[100%] Built target fzn
[100%] Built target fzn-parser_test
thejltres@thejltres-desktop:~/Documents/github/TheJltres/scheduler$ ll build/bin/ | grep scheduler
lrwxrwxrwx 1 thejltres thejltres   13 abr 19 17:26 scheduler -> scheduler-0.1*
-rwxrwxr-x 1 thejltres thejltres 193272 abr 19 17:26 scheduler-0.1*
```

15.5 Ejemplos de ejecución

Mostrar versión

Para mostrar la versión del ejecutable, tal y como estaba planeado, se puede mostrar de manera simple. Simplemente, se ha de introducir el comando a ejecutar y el parámetro '-v' o '--version' para mostrar la versión.

```
thejltres@thejltres-desktop:~/Documents/github/TheJltres/scheduler$ ./build/bin/scheduler -v
Scheduler version: 0.1
```

Mostrar ayuda

Se puede mostrar la ayuda del programa

```

thejltres@thejltres-desktop:~/Documents/github/TheJltres/scheduler$ ./build/bin/scheduler -h
Scheduler version: 0.1
Usage scheduler:
  scheduler -t <file> -s <file> [options]

Options:
  -t, --telescope <file>      Telescope configuration file
  -i, --import-objects <file>  File with objects to schedule

  --date <date>              Date of the observation (dd/MM/yyyy)
  --verbose                  Print all logs
  -h, --help                  Print help message
  -v, --version               Print version information

For more information, you can take a look at "man ./docs/scheduler"

```

Planificación básica

Se puede realizar una planificación básica, que permite planificar varios objetos de manera automática.

```

thejltres@thejltres-desktop:~/Documents/github/TheJltres/scheduler$ ./build/bin/scheduler -t tfrm -i objects.dat
Telescope
Date to schedule: 27/5/2024 18:40
Julian Date: 45438.3
Implementation of the Scheduler with OR-Tools
45437.8
45438.2
643
Object added: 1 - 859611 - 0 - 250 - 21
Object added: 1 - 861900 - 0 - 643 - 7
Object added: 1 - 834497 - 0 - 80 - 7
Object added: 1 - 928487 - 0 - 127 - 21
Object added: 1 - 964622 - 0 - 643 - 33
Object added: 1 - 857005 - 0 - 643 - 4
Object added: 1 - 846861 - 0 - 620 - 26
Object added: 1 - 832949 - 0 - 203 - 51
Object added: 1 - 807410 - 0 - 462 - 50
Object added: 1 - 808174 - 574 - 643 - 22
Object added: 1 - 927267 - 0 - 341 - 25
Object added: 1 - 936996 - 0 - 128 - 23
Object added: 1 - 904346 - 441 - 643 - 29
Object added: 1 - 983114 - 0 - 131 - 32
Object added: 1 - 800186 - 0 - 308 - 2
Object added: 1 - 946147 - 465 - 643 - 56
Object added: 1 - 815760 - 0 - 164 - 39
Object added: 1 - 798615 - 0 - 467 - 56
Solution found:
Telescope: 1 Job: 798615 Starts at: 221 until 277 - 56
Telescope: 1 Job: 800186 Starts at: 194 until 196 - 2
Telescope: 1 Job: 807410 Starts at: 277 until 327 - 50
Telescope: 1 Job: 808174 Starts at: 574 until 596 - 22
Telescope: 1 Job: 815760 Starts at: 83 until 122 - 39
Telescope: 1 Job: 832949 Starts at: 122 until 173 - 51
Telescope: 1 Job: 834497 Starts at: 0 until 7 - 7
Telescope: 1 Job: 846861 Starts at: 327 until 353 - 26
Telescope: 1 Job: 857005 Starts at: 393 until 397 - 4
Telescope: 1 Job: 859611 Starts at: 173 until 194 - 21
Telescope: 1 Job: 861900 Starts at: 386 until 393 - 7
Telescope: 1 Job: 904346 Starts at: 441 until 470 - 29
Telescope: 1 Job: 927267 Starts at: 196 until 221 - 25
Telescope: 1 Job: 928487 Starts at: 62 until 83 - 21
Telescope: 1 Job: 936996 Starts at: 39 until 62 - 23
Telescope: 1 Job: 946147 Starts at: 470 until 526 - 56
Telescope: 1 Job: 964622 Starts at: 353 until 386 - 33
Telescope: 1 Job: 983114 Starts at: 7 until 39 - 32
Optimal Schedule Length: 596

Statistics
CpSolverResponse summary:
status: OPTIMAL
objective: 596
best_bound: 596
integers: 19
booleans: 19
conflicts: 0
branches: 19
propagations: 0
integer_propagations: 103
restarts: 1
lp_iterations: 0
walltime: 0.00841672
usertime: 0.00841675
deterministic_time: 5.34692e-05
gap_integral: 0.000199077
solution_fingerprint: 0xfedd2fce011d70fe

```

17. Entrega y seguimiento

En esta primera entrega, se ha utilizado github como herramienta de publicación. Se han realizado pruebas manuales de un funcionamiento muy básico y se ha querido mostrar al usuario final un prototipo de aplicación que muestra las funcionalidades disponibles.

Desde el día de la entrega, se va a realizar una quedada virtual con el equipo del TFRM para explicar el proceso de compilación del programa y otros apartados más técnicos de la implementación. Se van a recopilar los comentarios de mejoras que respecten a nuevas funcionalidades o diferentes diseños de mostrado de los datos. Sin embargo, los errores siempre tendrán una mayor prioridad. La aplicación ha de poder funcionar sin lanzar excepciones no controladas, se han de controlar todos los posibles errores que puede haber y acabar el proceso de forma correcta.

En cada esprint de programación, se va a establecer una reunión que explique los cambios realizados y las mejoras implementadas, además de una guía de actualización en caso de modificar el comportamiento actual de la recogida de datos u otro apartado más estático.

Documentación de la entrega

Para realizar la entrega se ha utilizado a través de GitHub Releases, una funcionalidad dentro del repositorio en GitHub que permite gestionar y distribuir versiones específicas del proyecto.

Se ha asumido que el host destino tiene instalado y configurado cmake y se le han proporcionado los comandos utilizados en el desarrollo. Se ha comentado que idealmente, en la primera versión estable, el proyecto ha de tener instrucciones detalladas de la instalación y un archivo precompilado. También ha de ser compilado con los flags de Release para una mayor optimización.

Las opciones de ejecución del programa han sido las esperadas y se ha agradecido la documentación, ha sido muy sencillo encontrar las posibilidades del ejecutable. y entender su funcionamiento sin problemas. Sin embargo sí que falta documentación sobre los parámetros aceptados por algunos comandos, cómo el de objetos a observar y las características del telescopio. Es una funcionalidad que se ha de documentar para una comprensión más sencilla. Se ha acordado ponerlo en el manual del comando man.

Por último, se ha ejecutado el programa para realizar planificaciones, y cómo la primera versión de pruebas ha tenido un funcionamiento correcto, se han mostrado los objetos a observar en unos intervalos de tiempo coherentes. Se ha hecho un recordatorio sobre la aproximación modular para poder establecer

restricciones dinámicamente y se ha acordado que estará disponible en la versión final.

Creo que la entrega del proyecto ha sido muy satisfactoria y se espera poder seguir trabajando en el proyecto para añadir más funcionalidades y mejoras en el sistema de observación.

Conclusiones

A lo largo de este trabajo se ha podido realizar el estudio del funcionamiento básico de las planificaciones. Es un tema muy interesante, que se puede seguir estudiando a día de hoy y seguir ampliando nuevos horizontes. Sin embargo, en este proyecto no solo se han tenido que realizar las planificaciones, además, se han tenido que contemplar las restricciones de observación que puede tener un telescopio, y realizar adaptaciones para comprobar la visibilidad de los objetos espaciales en el telescopio. Ha sido todo un reto, tener en cuenta todos estos aspectos, sobre todo, dada mi limitada experiencia en el área. Aun así, he logrado realizar una base que puede facilitar las observaciones de un telescopio.

Se ha podido realizar una primera entrega del proyecto, que permitirá ver a los observadores del TFRM una interfaz de ejemplo para acceder al programa y programar observaciones ficticias que les permitirá ver todas las funcionalidades de la aplicación de manera sencilla. Aún quedan por añadir muchas funcionalidades y muchos retoques a la aplicación final, sin embargo, creemos que es un gran primer paso y se quieren añadir nuevas funcionalidades que simplifiquen las observaciones del telescopio.

El equipo de observadores del TFRM ha sido de gran ayuda para facilitar la comprensión de los aspectos más técnicos del telescopio y llevar a cabo los cálculos necesarios para las observaciones de forma adecuada.

Gracias a este proyecto, he tenido la oportunidad de realizar la implementación de un planificador para observar objetos en el espacio. Creo que ha sido una gran oportunidad para mí y mi carrera de programador. Espero poder mejorar el programa en el futuro y seguir trabajando con el equipo de observadores del TFRM en proyectos futuros.

Referencias

18. Glosario

TFRM. Acrónimo para Telescopio Fabra-ROA Montsec

ROA. Acrónimo para Real Instituto y Observatorio de la Armada

19. Bibliografía

TheJltres/scheduler [en línea] [fecha de consulta: 26 de marzo de 2024].

Disponible en: <https://github.com/TheJltres/scheduler>

Telescopio Fabra-ROA en el Montsec TFRM [en línea] [fecha de consulta: 8 de marzo de 2024]. Disponible en:

<https://observatorifabra.cat/astronomia/telescopio-fabra-roa-en-el-montsec-tfrm/?lang=es>

Scheduling (production processes) [en línea] [fecha de consulta: 14 de marzo de 2024] Disponible en:

[https://en.m.wikipedia.org/wiki/Scheduling_\(production_processes\)](https://en.m.wikipedia.org/wiki/Scheduling_(production_processes))

NP-Hard [en línea] [fecha de consulta: 14 de marzo de 2024] Disponible en:

<https://en.m.wikipedia.org/wiki/NP-hardness>

Optimal job scheduling [en línea] [fecha de consulta: 14 de marzo de 2024]

Disponible en: https://en.m.wikipedia.org/wiki/Optimal_job_scheduling

Search is dead, Long live proof [en línea] [fecha de consulta: 14 de marzo de 2024] Disponible en:

<https://people.eng.unimelb.edu.au/pstuckey/PPDP2013.pdf>

Constraint satisfisfaction problem [en línea] [fecha de consulta: 19 de marzo de 2024] Disponible en:

https://en.wikipedia.org/wiki/Constraint_satisfisfaction_problem

Boolean satisfiability problem [en línea] [fecha de consulta: 19 de marzo de 2024] Disponible en: https://en.wikipedia.org/wiki/Boolean_satisfiability_problem

Constraint programming [en línea] [fecha de consulta: 19 de marzo de 2024]

Disponible en: https://en.wikipedia.org/wiki/Constraint_programming

Google OR-Tools [en línea] [fecha de consulta: 2 de abril de 2024] Disponible en: <https://developers.google.com/optimization>

Awesome OR-Tools [en línea] [fecha de consulta: 2 de abril de 2024] Disponible en: https://github.com/or-tools/awesome_or-tools

IBM ILOG CPLEX Optimization Studio [en línea] [fecha de consulta: 2 de abril de 2024] Disponible en:

<https://www.ibm.com/products/ilog-cplex-optimization-studio>

Gurobi [en línea] [fecha de consulta: 2 de abril de 2024] Disponible en: <https://www.gurobi.com/>

OptaPlanner [en línea] [fecha de consulta: 2 de abril de 2024] Disponible en: <https://www.optaplanner.org/>

Observatory Control System [en línea] [fecha de consulta: 3 de abril de 2024] Disponible en: <https://observatorycontrolsystem.github.io/>

Arcsecond [en línea] [fecha de consulta: 3 de abril de 2024] Disponible en: <https://www.arcsecond.io/>

Search is Dead, Long Live Proof [en línea] [fecha de consulta: 7 de abril de 2024] Disponible en: <https://www.youtube.com/watch?v=IxiCHRFNgn0>

Argh! [en línea] [fecha de consulta: 15 de abril de 2024] Disponible en: <https://github.com/adishavit/argh>

mINI [en línea] [fecha de consulta: 15 de abril de 2024] Disponible en: <https://github.com/metayeti/mINI>

libastro [en línea] [fecha de consulta: 15 de abril de 2024] Disponible en: <https://github.com/XEphem/XEphem>

GNU Free Documentation License

Version 1.3, 3 November 2008

Copyright © 2000, 2001, 2002, 2007, 2008 Free Software Foundation, Inc.
<<https://fsf.org/>>

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

0. PREAMBLE

The purpose of this License is to make a manual, textbook, or other functional and useful document "free" in the sense of freedom: to assure everyone the effective freedom to copy and redistribute it, with or without modifying it, either commercially or noncommercially. Secondly, this License preserves for the author and publisher a way to get credit for their work, while not being considered responsible for modifications made by others.

This License is a kind of "copyleft", which means that derivative works of the document must themselves be free in the same sense. It complements the GNU General Public License, which is a copyleft license designed for free software.

We have designed this License in order to use it for manuals for free software, because free software needs free documentation: a free program should come with manuals providing the same freedoms that the software does. But this License is not limited to software manuals; it can be used for any textual work, regardless of subject matter or whether it is published as a printed book. We recommend this License principally for works whose purpose is instruction or reference.

1. APPLICABILITY AND DEFINITIONS

This License applies to any manual or other work, in any medium, that contains a notice placed by the copyright holder saying it can be distributed under the terms of this License. Such a notice grants a world-wide, royalty-free license, unlimited in duration, to use that work under the conditions stated herein. The "Document", below, refers to any such manual or work. Any member of the public is a licensee, and is addressed as "you". You accept the license if you copy, modify or distribute the work in a way requiring permission under copyright law.

A "Modified Version" of the Document means any work containing the Document or a portion of it, either copied verbatim, or with modifications and/or translated into another language.

A "Secondary Section" is a named appendix or a front-matter section of the Document that deals exclusively with the relationship of the publishers or authors of the Document to the Document's overall subject (or to related matters) and contains nothing that could fall directly within that overall subject. (Thus, if the Document is in part a textbook of mathematics, a Secondary Section may not explain any mathematics.) The relationship could be a matter of historical connection with the subject or with related matters, or of legal, commercial, philosophical, ethical or political position regarding them.

The "Invariant Sections" are certain Secondary Sections whose titles are designated, as being those of Invariant Sections, in the notice that says that the Document is released under this License. If a section does not fit the above definition of Secondary then it is not allowed to be designated as Invariant. The Document may contain zero Invariant Sections. If the Document does not identify any Invariant Sections then there are none.

The "Cover Texts" are certain short passages of text that are listed, as Front-Cover Texts or Back-Cover Texts, in the notice that says that the Document is released under this License. A Front-Cover Text may be at most 5 words, and a Back-Cover Text may be at most 25 words.

A "Transparent" copy of the Document means a machine-readable copy, represented in a format whose specification is available to the general public, that is suitable for revising the document straightforwardly with generic text editors or (for images composed of pixels) generic paint programs or (for drawings) some widely available drawing editor, and that is suitable for input to text formatters or for automatic translation to a variety of formats suitable for input to text formatters. A copy made in an otherwise Transparent file format whose markup, or absence of markup, has been arranged to thwart or discourage subsequent modification by readers is not Transparent. An image format is not Transparent if used for any substantial amount of text. A copy that is not "Transparent" is called "Opaque".

Examples of suitable formats for Transparent copies include plain ASCII without markup, Texinfo input format, LaTeX input format, SGML or XML using a publicly available DTD, and standard-conforming simple HTML, PostScript or PDF designed for human modification. Examples of transparent image formats include PNG, XCF and JPG. Opaque formats include proprietary formats that can be read and edited only by proprietary word processors, SGML or XML for which the DTD and/or processing tools are not generally available, and the

machine-generated HTML, PostScript or PDF produced by some word processors for output purposes only.

The "Title Page" means, for a printed book, the title page itself, plus such following pages as are needed to hold, legibly, the material this License requires to appear in the title page. For works in formats which do not have any title page as such, "Title Page" means the text near the most prominent appearance of the work's title, preceding the beginning of the body of the text.

The "publisher" means any person or entity that distributes copies of the Document to the public.

A section "Entitled XYZ" means a named subunit of the Document whose title either is precisely XYZ or contains XYZ in parentheses following text that translates XYZ in another language. (Here XYZ stands for a specific section name mentioned below, such as "Acknowledgements", "Dedications", "Endorsements", or "History".) To "Preserve the Title" of such a section when you modify the Document means that it remains a section "Entitled XYZ" according to this definition.

The Document may include Warranty Disclaimers next to the notice which states that this License applies to the Document. These Warranty Disclaimers are considered to be included by reference in this License, but only as regards disclaiming warranties: any other implication that these Warranty Disclaimers may have is void and has no effect on the meaning of this License.

2. VERBATIM COPYING

You may copy and distribute the Document in any medium, either commercially or noncommercially, provided that this License, the copyright notices, and the license notice saying this License applies to the Document are reproduced in all copies, and that you add no other conditions whatsoever to those of this License. You may not use technical measures to obstruct or control the reading or further copying of the copies you make or distribute. However, you may accept compensation in exchange for copies. If you distribute a large enough number of copies you must also follow the conditions in section 3.

You may also lend copies, under the same conditions stated above, and you may publicly display copies.

3. COPYING IN QUANTITY

If you publish printed copies (or copies in media that commonly have printed covers) of the Document, numbering more than 100, and the Document's license notice requires Cover Texts, you must enclose the copies in covers that carry, clearly and legibly, all these Cover Texts: Front-Cover Texts on the front

cover, and Back-Cover Texts on the back cover. Both covers must also clearly and legibly identify you as the publisher of these copies. The front cover must present the full title with all words of the title equally prominent and visible. You may add other material on the covers in addition. Copying with changes limited to the covers, as long as they preserve the title of the Document and satisfy these conditions, can be treated as verbatim copying in other respects.

If the required texts for either cover are too voluminous to fit legibly, you should put the first ones listed (as many as fit reasonably) on the actual cover, and continue the rest onto adjacent pages.

If you publish or distribute Opaque copies of the Document numbering more than 100, you must either include a machine-readable Transparent copy along with each Opaque copy, or state in or with each Opaque copy a computer-network location from which the general network-using public has access to download using public-standard network protocols a complete Transparent copy of the Document, free of added material. If you use the latter option, you must take reasonably prudent steps, when you begin distribution of Opaque copies in quantity, to ensure that this Transparent copy will remain thus accessible at the stated location until at least one year after the last time you distribute an Opaque copy (directly or through your agents or retailers) of that edition to the public.

It is requested, but not required, that you contact the authors of the Document well before redistributing any large number of copies, to give them a chance to provide you with an updated version of the Document.

4. MODIFICATIONS

You may copy and distribute a Modified Version of the Document under the conditions of sections 2 and 3 above, provided that you release the Modified Version under precisely this License, with the Modified Version filling the role of the Document, thus licensing distribution and modification of the Modified Version to whoever possesses a copy of it. In addition, you must do these things in the Modified Version:

- A. Use in the Title Page (and on the covers, if any) a title distinct from that of the Document, and from those of previous versions (which should, if there were any, be listed in the History section of the Document). You may use the same title as a previous version if the original publisher of that version gives permission.
- B. List on the Title Page, as authors, one or more persons or entities responsible for authorship of the modifications in the Modified Version, together with at least five of the principal authors of the Document (all of

its principal authors, if it has fewer than five), unless they release you from this requirement.

- C. State on the Title page the name of the publisher of the Modified Version, as the publisher.
- D. Preserve all the copyright notices of the Document.
- E. Add an appropriate copyright notice for your modifications adjacent to the other copyright notices.
- F. Include, immediately after the copyright notices, a license notice giving the public permission to use the Modified Version under the terms of this License, in the form shown in the Addendum below.
- G. Preserve in that license notice the full lists of Invariant Sections and required Cover Texts given in the Document's license notice.
- H. Include an unaltered copy of this License.
- I. Preserve the section Entitled "History", Preserve its Title, and add to it an item stating at least the title, year, new authors, and publisher of the Modified Version as given on the Title Page. If there is no section Entitled "History" in the Document, create one stating the title, year, authors, and publisher of the Document as given on its Title Page, then add an item describing the Modified Version as stated in the previous sentence.
- J. Preserve the network location, if any, given in the Document for public access to a Transparent copy of the Document, and likewise the network locations given in the Document for previous versions it was based on. These may be placed in the "History" section. You may omit a network location for a work that was published at least four years before the Document itself, or if the original publisher of the version it refers to gives permission.
- K. For any section Entitled "Acknowledgements" or "Dedications", Preserve the Title of the section, and preserve in the section all the substance and tone of each of the contributor acknowledgements and/or dedications given therein.
- L. Preserve all the Invariant Sections of the Document, unaltered in their text and in their titles. Section numbers or the equivalent are not considered part of the section titles.
- M. Delete any section Entitled "Endorsements". Such a section may not be included in the Modified Version.
- N. Do not retitle any existing section to be Entitled "Endorsements" or to conflict in title with any Invariant Section.
- O. Preserve any Warranty Disclaimers.

If the Modified Version includes new front-matter sections or appendices that qualify as Secondary Sections and contain no material copied from the Document, you may at your option designate some or all of these sections as invariant. To do this, add their titles to the list of Invariant Sections in the

Modified Version's license notice. These titles must be distinct from any other section titles.

You may add a section Entitled "Endorsements", provided it contains nothing but endorsements of your Modified Version by various parties—for example, statements of peer review or that the text has been approved by an organization as the authoritative definition of a standard.

You may add a passage of up to five words as a Front-Cover Text, and a passage of up to 25 words as a Back-Cover Text, to the end of the list of Cover Texts in the Modified Version. Only one passage of Front-Cover Text and one of Back-Cover Text may be added by (or through arrangements made by) any one entity. If the Document already includes a cover text for the same cover, previously added by you or by arrangement made by the same entity you are acting on behalf of, you may not add another; but you may replace the old one, on explicit permission from the previous publisher that added the old one.

The author(s) and publisher(s) of the Document do not by this License give permission to use their names for publicity for or to assert or imply endorsement of any Modified Version.

5. COMBINING DOCUMENTS

You may combine the Document with other documents released under this License, under the terms defined in section 4 above for modified versions, provided that you include in the combination all of the Invariant Sections of all of the original documents, unmodified, and list them all as Invariant Sections of your combined work in its license notice, and that you preserve all their Warranty Disclaimers.

The combined work need only contain one copy of this License, and multiple identical Invariant Sections may be replaced with a single copy. If there are multiple Invariant Sections with the same name but different contents, make the title of each such section unique by adding at the end of it, in parentheses, the name of the original author or publisher of that section if known, or else a unique number. Make the same adjustment to the section titles in the list of Invariant Sections in the license notice of the combined work.

In the combination, you must combine any sections Entitled "History" in the various original documents, forming one section Entitled "History"; likewise combine any sections Entitled "Acknowledgements", and any sections Entitled "Dedications". You must delete all sections Entitled "Endorsements".

6. COLLECTIONS OF DOCUMENTS

You may make a collection consisting of the Document and other documents released under this License, and replace the individual copies of this License in the various documents with a single copy that is included in the collection, provided that you follow the rules of this License for verbatim copying of each of the documents in all other respects.

You may extract a single document from such a collection, and distribute it individually under this License, provided you insert a copy of this License into the extracted document, and follow this License in all other respects regarding verbatim copying of that document.

7. AGGREGATION WITH INDEPENDENT WORKS

A compilation of the Document or its derivatives with other separate and independent documents or works, in or on a volume of a storage or distribution medium, is called an "aggregate" if the copyright resulting from the compilation is not used to limit the legal rights of the compilation's users beyond what the individual works permit. When the Document is included in an aggregate, this License does not apply to the other works in the aggregate which are not themselves derivative works of the Document.

If the Cover Text requirement of section 3 is applicable to these copies of the Document, then if the Document is less than one half of the entire aggregate, the Document's Cover Texts may be placed on covers that bracket the Document within the aggregate, or the electronic equivalent of covers if the Document is in electronic form. Otherwise they must appear on printed covers that bracket the whole aggregate.

8. TRANSLATION

Translation is considered a kind of modification, so you may distribute translations of the Document under the terms of section 4. Replacing Invariant Sections with translations requires special permission from their copyright holders, but you may include translations of some or all Invariant Sections in addition to the original versions of these Invariant Sections. You may include a translation of this License, and all the license notices in the Document, and any Warranty Disclaimers, provided that you also include the original English version of this License and the original versions of those notices and disclaimers. In case of a disagreement between the translation and the original version of this License or a notice or disclaimer, the original version will prevail.

If a section in the Document is Entitled "Acknowledgements", "Dedications", or "History", the requirement (section 4) to Preserve its Title (section 1) will typically require changing the actual title.

9. TERMINATION

You may not copy, modify, sublicense, or distribute the Document except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, or distribute it is void, and will automatically terminate your rights under this License.

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, receipt of a copy of some or all of the same material does not give you any rights to use it.

10. FUTURE REVISIONS OF THIS LICENSE

The Free Software Foundation may publish new, revised versions of the GNU Free Documentation License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. See <https://www.gnu.org/licenses/>.

Each version of the License is given a distinguishing version number. If the Document specifies that a particular numbered version of this License "or any later version" applies to it, you have the option of following the terms and conditions either of that specified version or of any later version that has been published (not as a draft) by the Free Software Foundation. If the Document does not specify a version number of this License, you may choose any version ever published (not as a draft) by the Free Software Foundation. If the Document specifies that a proxy can decide which future versions of this License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Document.

11. RELICENSING

"Massive Multiauthor Collaboration Site" (or "MMC Site") means any World Wide Web server that publishes copyrightable works and also provides prominent facilities for anybody to edit those works. A public wiki that anybody can edit is an example of such a server. A "Massive Multiauthor Collaboration"

(or "MMC") contained in the site means any set of copyrightable works thus published on the MMC site.

"CC-BY-SA" means the Creative Commons Attribution-Share Alike 3.0 license published by Creative Commons Corporation, a not-for-profit corporation with a principal place of business in San Francisco, California, as well as future copyleft versions of that license published by that same organization.

"Incorporate" means to publish or republish a Document, in whole or in part, as part of another Document.

An MMC is "eligible for relicensing" if it is licensed under this License, and if all works that were first published under this License somewhere other than this MMC, and subsequently incorporated in whole or in part into the MMC, (1) had no cover texts or invariant sections, and (2) were thus incorporated prior to November 1, 2008.

The operator of an MMC Site may republish an MMC contained in the site under CC-BY-SA on the same site at any time before August 1, 2009, provided the MMC is eligible for relicensing.